ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ



BÁO CÁO CUỐI KỲ

TỔNG KẾT DỰ ÁN NHÓM – ĐÁNH GIÁ KẾT THÚC HỌC PHẦN

ET4710 – LẬP TRÌNH ỨNG DỤNG DI ĐỘNG

ROOMHUB
ỨNG DỤNG ĐẶT VÀ QUẢN LÝ PHÒNG HỌC, PHÒNG HỌP

Nền tảng: React Native Community CLI và TypeScript

Nhóm thực hiện:	RoomHub
Nhóm trưởng:	Đỗ Thùy Linh
Lớp cao học:	KH2025B
Lớp học phần:	ET4710-1-26 (26.1A01)
Đợt học / năm:	2026
Giảng viên:	PGS.TS. Đỗ Trọng Tuấn

Đỗ Thùy Linh	20252478M
Hoàng Văn Tùng	20252389M
Nguyễn Việt Dũng	20252690M
Nguyễn Đình Phúc	20251012M

HÀ NỘI, NĂM 2026

PHIẾU THÔNG TIN NHÓM VÀ DỰ ÁN

Bảng 1: Thông tin sản phẩm và bản mã nguồn

Hạng mục	Thông tin
Ứng dụng	RoomHub – đặt và quản lý phòng học, phòng họp.
Nền tảng	React Native 0.86.2, React 19.2.3, TypeScript 5.9.3; Community CLI 20.1.0. Nền tảng triển khai: Android.
Repository	https://github.com/xwomen1/classroom-booking
Nhánh mã nguồn	*main*.
Bản demo	Ảnh thực nghiệm và giải thích giao diện ở Chương 4; hướng dẫn tái lập Android Release trong README và Phụ lục A.
Dữ liệu	Async Storage local hoặc REST tới Spring Boot/H2; hai kho độc lập. OneIoT/MQTT là kết nối riêng.
Nhóm trưởng	Đỗ Thùy Linh.

Bảng 2: Thành viên, phần việc và tỷ lệ đóng góp

Thành viên / mã học viên	Hạng mục phụ trách	Đóng góp
Đỗ Thùy Linh / 20252478M	API, đồng bộ dữ liệu, lọc phòng, tách controller và tiện ích thời gian.	26%
Hoàng Văn Tùng / 20252389M	Khung app, các màn hình và nghiệp vụ nền, SmartLock/native, tích hợp và sửa hồi quy.	28%
Nguyễn Việt Dũng / 20252690M	Tự động duyệt và bộ kiểm thử chất lượng.	26%
Nguyễn Đình Phúc / 20251012M	Đặt lặp, vắng mặt, giao diện duyệt và sơ đồ thiết kế.	20%
Tổng	Đóng góp chung của nhóm.	100%

Cam kết của nhóm

Nhóm chúng tôi cam kết dự án do nhóm thực hiện; nội dung, kết quả, hình ảnh và số liệu trong báo cáo phản ánh trung thực quá trình thực hiện. Đóng góp của các thành viên, nguồn tài liệu tham khảo và công cụ hỗ trợ được ghi nhận đầy đủ. Nhóm chịu trách nhiệm về nội dung báo cáo.

Đại diện nhóm:	**Ngày:**	**Chữ ký / xác nhận:**
Hoàng Văn Tùng	08/11/2026	

TÓM TẮT DỰ ÁN

RoomHub là ứng dụng hỗ trợ đặt và quản lý phòng học, phòng họp, nhằm tập trung việc đăng ký sử dụng, phê duyệt và bàn giao quyền tiếp cận phòng. Ứng dụng hướng đến giảng viên, cán bộ, người học và quản trị viên cơ sở vật chất, giúp người dùng theo dõi lịch phòng và giảm xung đột khi tổ chức hoạt động học tập, làm việc.

Người dùng có thể tra cứu phòng theo thời gian, tầng, sức chứa và thiết bị; đặt phòng một lần hoặc lặp theo tuần; theo dõi, chỉnh sửa và hủy yêu cầu. Quản trị viên quản lý tài khoản, phòng, quy định đặt phòng và bảo trì; xử lý yêu cầu bằng phê duyệt thủ công hoặc tự động. Quy trình tiếp cận phòng hỗ trợ hẹn nhận chìa, ủy quyền hoặc tạo mã PIN tạm thời; các chức năng điểm danh, ghi chú và thông báo giúp theo dõi quá trình sử dụng.

Giải pháp sử dụng React Native và TypeScript cho ứng dụng Android, Async Storage cho dữ liệu local và Spring Boot/H2 cho chế độ REST. Sản phẩm đã hoàn thiện các luồng nghiệp vụ chính, vận hành trên Android và kết nối SmartLock trong thử nghiệm thực tế. Hướng phát triển tiếp theo là mở rộng quy mô quản lý và trải nghiệm sử dụng.

Từ khóa: React Native; TypeScript; đặt phòng; quản lý phòng; SmartLock.

THUẬT NGỮ VÀ TỪ VIẾT TẮT

Bảng 3: Thuật ngữ sử dụng trong báo cáo

Thuật ngữ	Ý nghĩa trong ngữ cảnh dự án
AES / CTR	Mã hóa payload PIN trong module MQTT native.
API	Hợp đồng gọi dịch vụ; dự án dùng HTTP API của booking-server.
APK	Gói cài Android; bản Release chứa JavaScript bundle.
BR / FR / NFR	Quy tắc nghiệp vụ / yêu cầu chức năng / yêu cầu phi chức năng.
CRUD	Tạo, đọc, sửa và xóa dữ liệu theo quyền.
DI / IoC	Spring quản lý thành phần và truyền phụ thuộc cho filter/controller/service.
DTO	Dữ liệu vào/ra dịch vụ; khác entity lưu trong H2.
HTTP / REST	Giao tiếp API theo tài nguyên/phương thức, dùng JSON và Bearer token.
IoT / oneM2M	Internet vạn vật / quy ước bản tin và chủ đề dùng với OneIoT.
JPA / ORM	Ánh xạ entity Java tới dữ liệu quan hệ.
JSON	Định dạng dữ liệu API, kho local và các trường lồng của booking.
JVM	Máy ảo chạy backend Java 17.
MQTT	Giao thức publish/subscribe qua broker; kết nối khóa từ native Android.
PIN	Mã tạm bảy chữ số theo quyền phòng, booking và khoảng hiệu lực.
PUBACK / QoS	Phản hồi broker khi nhận publish / mức chất lượng giao nhận MQTT.
RN / TS	React Native / TypeScript; nền tảng giao diện và ngôn ngữ app.
TLS	Bảo vệ kênh truyền; native hiện còn hạn chế xác minh chứng thư.
UC / UML	Ca sử dụng / ký pháp mô hình hóa phân tích và thiết kế.
UUID	Định danh duy nhất dạng chuỗi.
Local / REST	Async Storage trên máy / API tới backend; hai kho không tự đồng bộ.

MỤC LỤC

PHIẾU THÔNG TIN NHÓM VÀ DỰ ÁN i

TÓM TẮT DỰ ÁN ii

THUẬT NGỮ VÀ TỪ VIẾT TẮT ii

1 GIỚI THIỆU DỰ ÁN 1

1.1 Bối cảnh và bài toán thực tế . . . 1
1.2 Mục tiêu và tiêu chí nghiệm thu . . . 1
1.3 Đối tượng, phạm vi và giả định . . . 1
1.4 Sản phẩm bàn giao . . . 1

2 KHẢO SÁT VÀ ĐẶC TẢ YÊU CẦU 2

2.1 Bối cảnh khảo sát và mô hình nghiệp vụ . . . 2
2.2 Tác nhân, kịch bản sử dụng và luồng nghiệp vụ . . . 2
2.3 Yêu cầu chức năng . . . 8
2.4 Quy tắc nghiệp vụ và các trường hợp biên . . . 8
2.5 Đặc tả các use case . . . 9
2.6 Yêu cầu phi chức năng . . . 16
2.7 Tiêu chí chấp nhận và mức độ ưu tiên . . . 17

3 THIẾT KẾ HỆ THỐNG 18

3.1 Lựa chọn công nghệ và lập luận . . . 18
3.2 Kiến trúc và luồng dữ liệu . . . 19
3.3 Mô hình dữ liệu và giao tiếp dịch vụ . . . 23
3.4 Thiết kế giao diện và điều hướng . . . 25
3.5 Dữ liệu cá nhân, quyền thiết bị và các ràng buộc . . . 26
3.6 Các giải pháp kỹ thuật cho luồng nghiệp vụ cốt lõi . . . 27

4 TRIỂN KHAI ỨNG DỤNG **31**
4.1 Môi trường và cấu trúc mã nguồn . . . 31
4.2 Ánh xạ yêu cầu chức năng tới mã nguồn . . . 32
4.3 Ma trận kỹ thuật React Native . . . 33
4.4 Phân tích hai đoạn mã cốt lõi . . . 34
4.5 Truy vết luồng đăng nhập và xử lý ngoại lệ . . . 36
4.6 Giao diện và sản phẩm chạy thử . . . 36
4.7 Cấu hình, cài đặt và tái lập bản chạy . . . 39

5 KIỂM THỬ VÀ ĐÁNH GIÁ **40**
5.1 Cấu hình và phương pháp kiểm thử . . . 40
5.2 Ca kiểm thử chức năng và kết quả thực tế . . . 41
5.3 Phạm vi đánh giá chất lượng định lượng . . . 44
5.4 Đối chiếu yêu cầu, hạn chế và rủi ro . . . 44

6 TỔ CHỨC NHÓM VÀ TỔNG KẾT **45**
6.1 Quá trình thực hiện và các mốc chính . . . 45
6.2 Phân công và đóng góp . . . 45
6.3 Kết quả so với mục tiêu . . . 45
6.4 Bài học kỹ thuật và hướng phát triển . . . 46

TÀI LIỆU THAM KHẢO **47**

A DANH MỤC MINH CHỨNG VÀ DEMO **48**
A.1 Danh mục minh chứng . . . 48
A.2 Kịch bản trình diễn 3–7 phút . . . 49
A.3 Mô hình dữ liệu và nguồn sơ đồ chi tiết . . . 49
A.4 Quy trình tái lập kỹ thuật . . . 51

B ĐỐI CHIẾU ĐÓNG GÓP THÀNH VIÊN **52**
B.1 Đối chiếu từng thành viên . . . 52
B.2 Tái lập thống kê . . . 52
B.3 Công cụ hỗ trợ . . . 52

DANH MỤC HÌNH

2.1 Các use case chung và hai vai trò của RoomHub . . . 3
2.2 Use case của người dùng và tương tác với hệ thống khóa . . . 3
2.3 Use case quản trị tài nguyên và xử lý yêu cầu . . . 4
2.4 Luồng tạo yêu cầu và quyết định duyệt trong nhánh REST . . . 5
2.5 Luồng tạo PIN, chờ gửi và giao tiếp OneIoT . . . 6
2.6 Thỏa thuận nhận khóa, xác nhận có mặt và xử lý bảo trì . . . 7
2.7 Vòng đời booking và các mốc sử dụng liên quan . . . 8
3.1 Các thành phần chạy: app Android, REST backend và OneIoT . . . 19
3.2 Trách nhiệm và các phụ thuộc chính giữa module React Native . . . 20
3.3 Các lớp giao tiếp, nghiệp vụ và lưu trữ của backend . . . 21
3.4 Bố trí triển khai ứng dụng Android, backend và kết nối SmartLock . . . 22
3.5 Luồng màn hình và các tác vụ chính của người dùng . . . 25
3.6 Luồng màn hình quản trị và các điểm vào từ thông báo . . . 26
3.7 Trình tự tạo booking qua REST và nhánh tự duyệt/thủ công . . . 28
3.8 Tạo, lưu và gửi PIN; xác nhận publish của broker tách với thực thi trên khóa . . . 29
4.1 Đăng nhập, chức năng User và Admin (trái sang phải) . . . 37
4.2 Chọn tiêu chí, chọn phòng và gửi yêu cầu (trái sang phải) . . . 37
4.3 Danh sách tầng và phòng khi mở tầng (trái sang phải) . . . 38
4.4 Tham số đặt phòng (trái), công tắc thông báo và tự duyệt (phải) . . . 38
A.1 Mô hình thực thể và quan hệ logic của RoomHub . . . 50

DANH MỤC BẢNG

1 Thông tin sản phẩm và bản mã nguồn i
2 Thành viên, phần việc và tỷ lệ đóng góp i
3 Thuật ngữ sử dụng trong báo cáo ii
1.1 Mục tiêu, cách xác minh và mức kết quả 1
2.1 Hai phương thức tiếp cận được model hỗ trợ 2
2.2 Yêu cầu chức năng và minh chứng kiểm thử 8
2.3 Quy tắc nghiệp vụ đối chiếu với mã nguồn 8
2.4 Tiêu chí phi chức năng và giới hạn đánh giá 16
2.5 Điều kiện chấp nhận chức năng 17
3.1 Công nghệ, lý do lựa chọn và hạn chế kỹ thuật 18
3.2 Trách nhiệm của 11 module nghiệp vụ 21
3.3 Trách nhiệm các lớp Java và dữ liệu trao đổi 22
3.4 Mười một thực thể và quan hệ nghiệp vụ chính 23
3.5 Tổng hợp endpoint, dữ liệu và quyền truy cập 24
3.6 Màn hình, mục đích và dữ liệu điều hướng 26
3.7 Thứ tự tạo booking trong giao dịch Java 27
3.8 Đối chiếu thao tác local và REST 30
4.1 Công cụ và cấu hình biên dịch 31
4.2 Cấu trúc thư mục và trách nhiệm 31
4.3 Ánh xạ chức năng, vị trí và bằng chứng 32
4.4 Kỹ thuật, cách áp dụng, lý do và kiểm chứng 33
4.5 Từ thao tác tới render trong luồng đăng nhập 36
4.6 Chế độ chạy, phụ thuộc và phạm vi dữ liệu 39
5.1 Môi trường, dữ liệu và phạm vi kết quả 40
5.2 Phương pháp và vai trò trong kiểm thử 40
5.3 Ca tiêu biểu, kỳ vọng, thực tế và kết luận 41
5.4 Bước thử, kỳ vọng và quan sát trên máy thật 42
5.5 Ca nghiệp vụ chi tiết và kết quả 42
5.6 Phương pháp và phạm vi đánh giá định lượng 44
6.1 Giai đoạn thực hiện và sản phẩm 45
6.2 Mã nguồn, kiểm thử và tỷ lệ đóng góp 45
A.1 Hồ sơ bàn giao và vị trí 48
A.2 Kịch bản demo và kết quả mong đợi 49
B.1 Đầu việc, sản phẩm và minh chứng thành viên 52

CHƯƠNG 1. GIỚI THIỆU DỰ ÁN

1.1 Bối cảnh và bài toán thực tế

RoomHub liên kết đặt phòng, phê duyệt và tiếp cận phòng theo vai trò user/admin, thay cho đối chiếu lịch và bàn giao khóa rời rạc qua bảng tính hoặc trao đổi trực tiếp [2]. Khóa cơ/thẻ có hẹn nhận, ủy quyền và điểm danh; khóa số có quyền PIN theo thời hạn; người đang dùng phòng được báo tình trạng/bảo trì.

C7, Đại học Bách khoa Hà Nội là ví dụ tham chiếu của đề xuất. Dữ liệu minh họa do nhóm xây dựng gồm tám tầng, bảy phòng mỗi tầng, cùng thông tin sức chứa và thiết bị.

1.2 Mục tiêu và tiêu chí nghiệm thu

Bảng 1.1: Mục tiêu, cách xác minh và mức kết quả

Mã / mục tiêu	**Cách kiểm chứng**	**Kết quả có căn cứ**
G01 – Đặt phòng hợp lệ	Đăng nhập, lọc phòng, gửi yêu cầu; test trùng lịch, bảo trì và hạn mức.	Luồng tìm, đặt phòng và các quy tắc xung đột, bảo trì, hạn mức hoạt động đúng trong ứng dụng.
G02 – Duyệt và tiếp cận	Duyệt tay/tự động, quyền PIN theo phòng, nhận chìa, no-show.	Hoàn thành luồng duyệt, cấp quyền và sử dụng PIN với SmartLock trong chạy thử thực tế.
G03 – Trải nghiệm Android	Bản Release chạy không cần Metro; màn hình theo vai trò và thông báo đúng người nhận.	Ứng dụng Android chạy độc lập Metro, hiển thị chức năng đúng vai trò và xử lý thông báo.
G04 – Tái lập và phối hợp	README, lockfile, test, lịch sử Git và minh chứng từng thành viên.	README mô tả cài đặt và cấu hình; lockfile cố định phụ thuộc; bộ kiểm thử và hồ sơ bàn giao hỗ trợ tái lập bản chạy Android.

1.3 Đối tượng, phạm vi và giả định

Người dùng mục tiêu là giảng viên, cán bộ hoặc người học; vai trò user đại diện cho nhóm đặt và sử dụng phòng, còn admin quản lý tài nguyên. Giao diện dùng nhãn GIẢNG VIÊN cho vai trò user theo bối cảnh phòng học của dữ liệu thử. Phạm vi gồm tài khoản/hồ sơ, phòng, booking một lần/lặp tuần, duyệt, PIN/hẹn chìa, điểm danh, bảo trì, ghi chú, thông báo và cấu hình.

Giả định đồng hồ đúng, phòng đã khai báo và có một loại khóa. Mặc định đặt trước 1–3 ngày, tối đa 2 lượt hoạt động; admin đổi được. REST cần server/URL khả dụng; local không hòa giải dữ liệu với server. Mở rộng nhiều tòa nhà/khóa, server push và tự động hóa quản lý PIN là các hướng phát triển tiếp theo.

1.4 Sản phẩm bàn giao

Sản phẩm bàn giao gồm ứng dụng Android để đặt và quản lý phòng; mã nguồn ứng dụng và backend Java để cài đặt, vận hành và phát triển tiếp; README hướng dẫn cấu hình, build và chạy thử; dữ liệu mẫu, bộ kiểm thử, ảnh giao diện, sơ đồ và báo cáo. Mã nguồn được lưu tại nhánh main của repository nhóm [3]. Danh mục minh chứng và kịch bản demo nằm ở Phụ lục A; phần việc thành viên ở Phụ lục B.

CHƯƠNG 2. KHẢO SÁT VÀ ĐẶC TẢ YÊU CẦU

2.1 Bối cảnh khảo sát và mô hình nghiệp vụ

2.1.1 Quy trình tham chiếu và phạm vi dữ liệu

Phương pháp khảo sát là phân tích quy trình nghiệp vụ tham chiếu của nhóm [2], từ đó xác định tác nhân, trường hợp sử dụng, trạng thái và quyền thao tác. Dữ liệu phòng do nhóm xây dựng để mô hình hóa tình huống; đây là khảo sát quy trình và phân tích yêu cầu, không phải nghiên cứu thống kê người dùng.

Hai vai trò tài khoản là *user* và *admin*; OneIoT là hệ thống khóa bên ngoài. Dữ liệu tám tầng/bảy phòng mỗi tầng phục vụ kiểm tra giao diện và lọc phòng.

2.1.2 Phương thức tiếp cận phòng trong bản triển khai

Mỗi phòng có một loại khóa *PHYSICAL_KEY* hoặc *PIN_CODE* (Bảng 2.1); người đặt không tự đổi phương thức khóa cho một lượt.

Bảng 2.1: Hai phương thức tiếp cận được model hỗ trợ

Loại khóa	Luồng sau khi duyệt	Đặc điểm vận hành
Khóa cơ/thẻ	User đề xuất giờ nhận; admin chấp nhận hoặc đề xuất lại; có thể khai báo người nhận hộ. Admin xác nhận có mặt trong giờ sử dụng.	Lịch hẹn và ủy quyền hỗ trợ bàn giao chìa; việc trả chìa được thực hiện theo quy trình quản lý phòng.
Khóa số	Phòng được gắn SmartLock; người có quyền tạo PIN theo booking và thời hạn; mã chờ hoặc được gửi qua OneIoT.	Ứng dụng lưu PIN, gửi lệnh qua OneIoT và tiếp nhận sự kiện sử dụng khóa theo phiên kết nối.

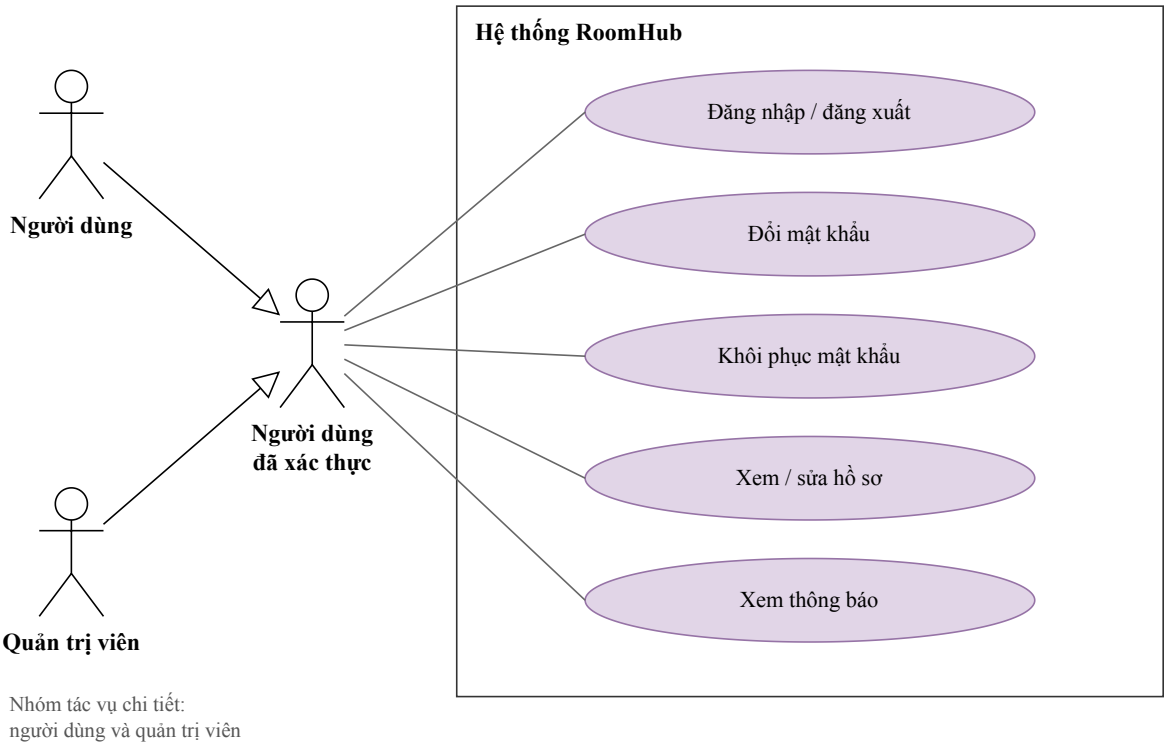
2.2 Tác nhân, kịch bản sử dụng và luồng nghiệp vụ

2.2.1 Phạm vi của từng tác nhân

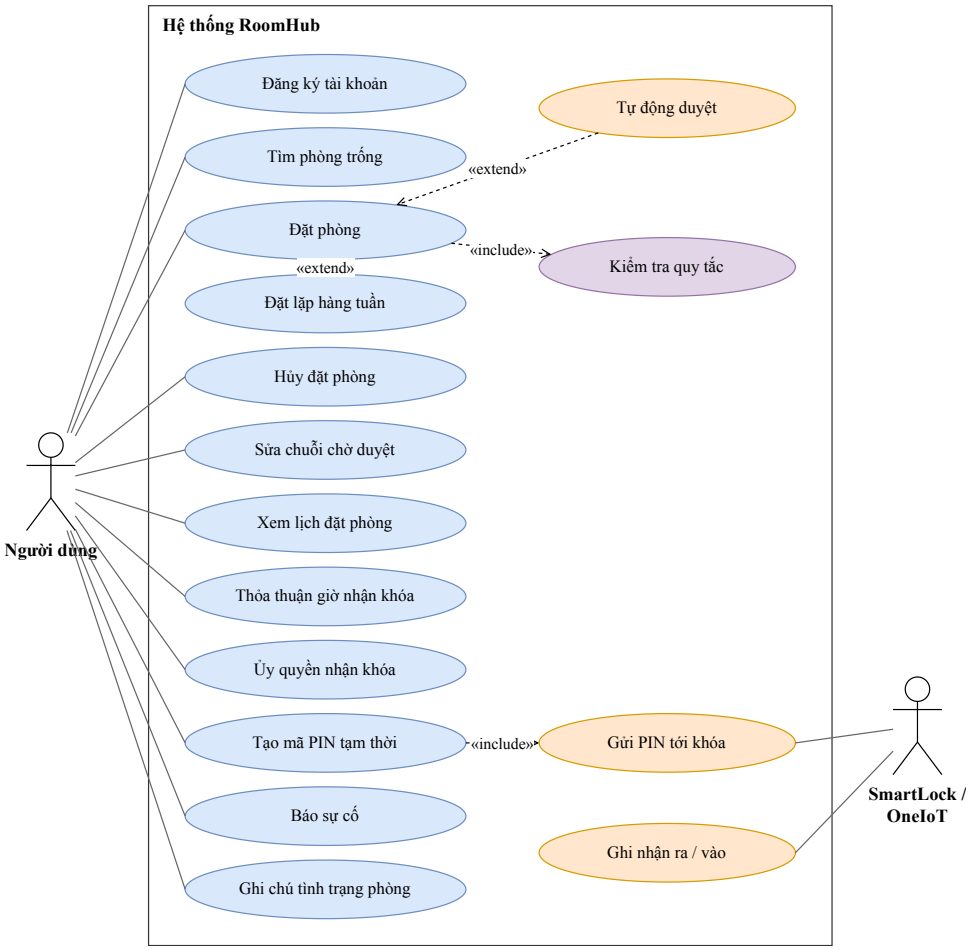
Hình 2.1 mô tả chức năng chung: user quản lý lượt của mình, admin xử lý yêu cầu/tài nguyên. Repository local hoặc filter/service server tiếp tục kiểm quyền sau UI.

Hình 2.2 tách tác vụ user. Mọi yêu cầu đặt đều được kiểm quy tắc trước nhánh tự duyệt, không do user tự cấp quyền.

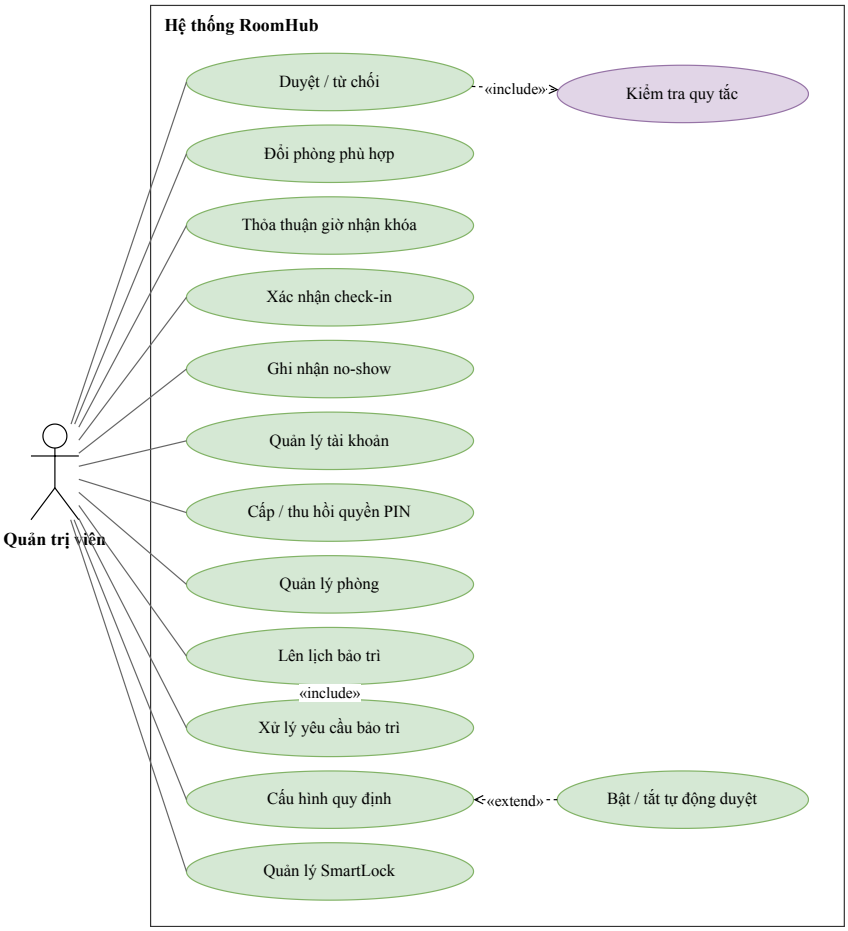
Hình 2.3 mô tả tác vụ admin; cấp/thu hồi quyền PIN theo cặp user–phòng không đổi vai trò tài khoản.



Hình 2.1: Các use case chung và hai vai trò của RoomHub



Hình 2.2: Use case của người dùng và tương tác với hệ thống khóa



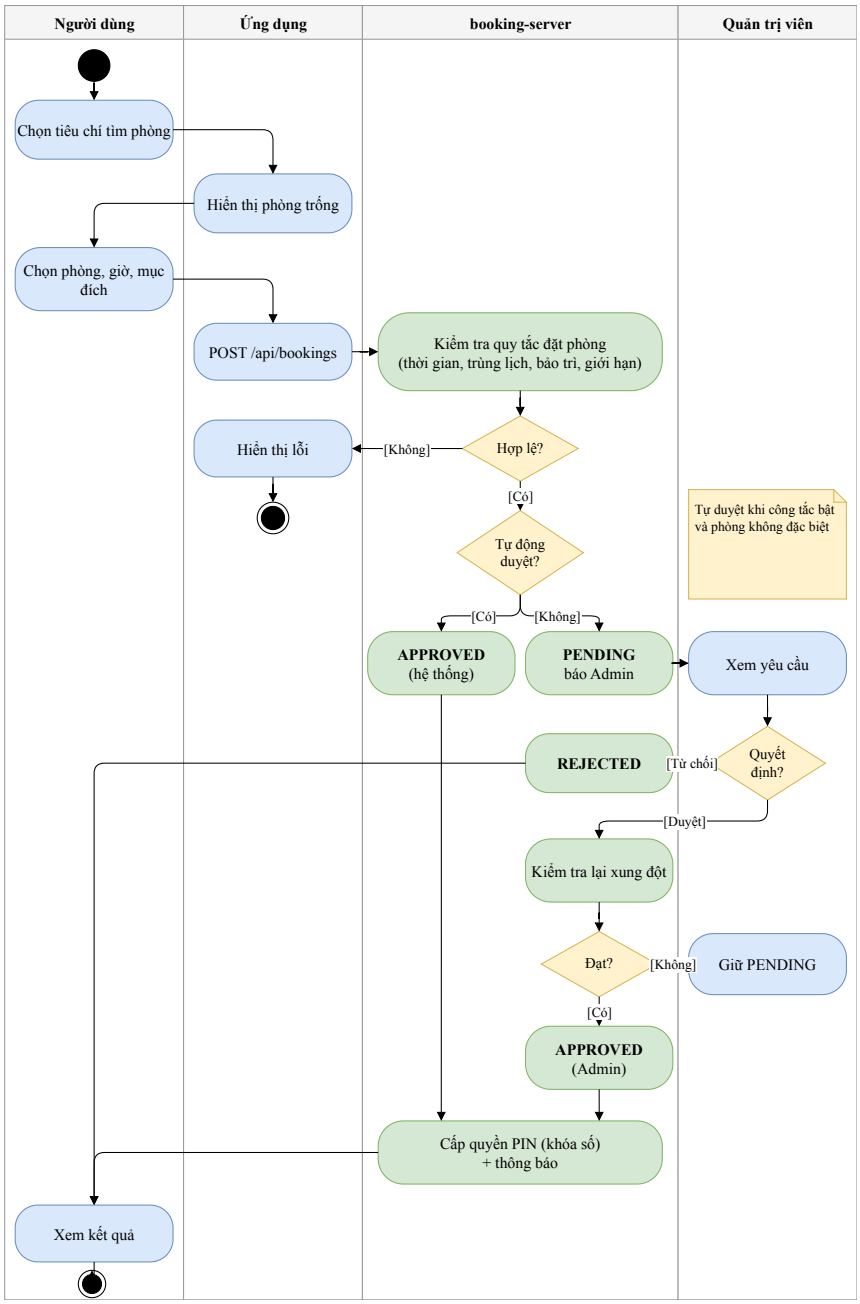
Hình 2.3: Use case quản trị tài nguyên và xử lý yêu cầu

2.2.2 Ba kịch bản nghiệp vụ tiêu biểu

Ba tuyến nghiệp vụ là: đặt/duyệt phòng PIN (UC02, UC03, UC05); nhận chìa, ủy quyền và điểm danh/no-show (UC07, UC12); ghi chú và xử lý bảo trì trong giờ dùng (UC08, UC13). Điều kiện và ngoại lệ được đặc tả tại mục 2.5.

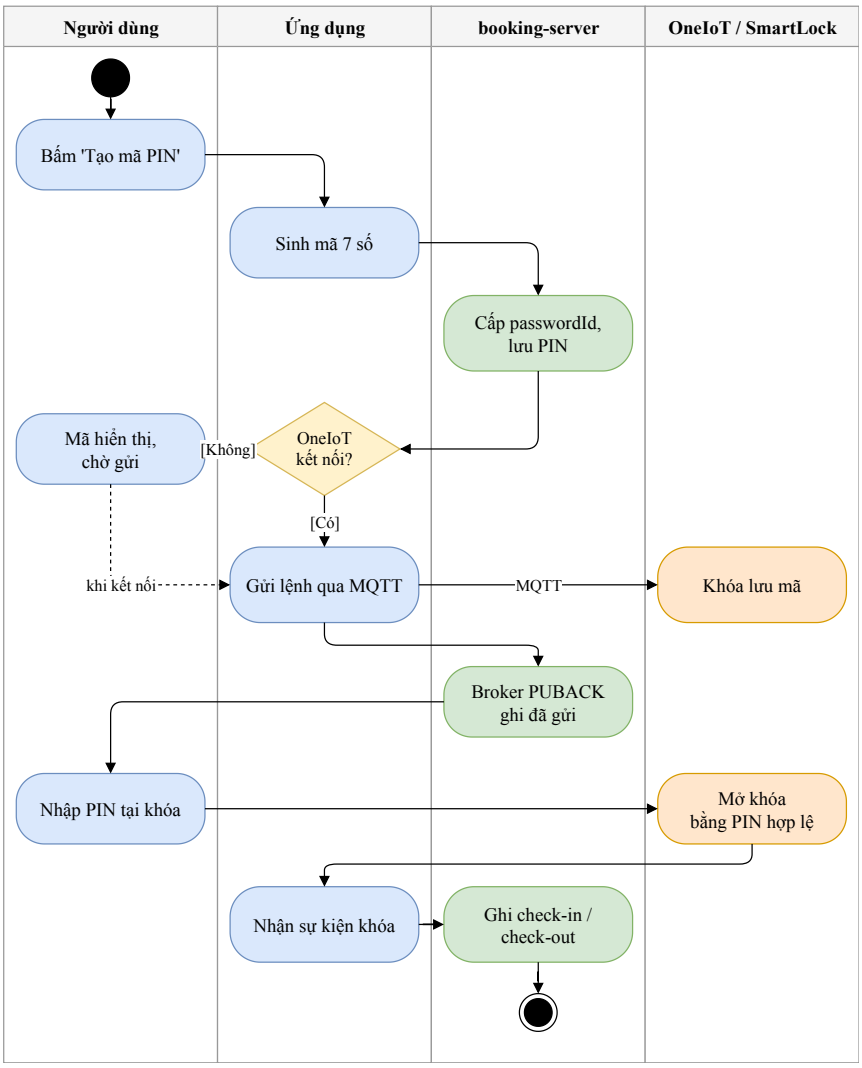
2.2.3 Đặt phòng, tự duyệt và cấp quyền tiếp cận

Hình 2.4 thể hiện nhánh REST: UI gửi API, server kiểm điều kiện trước khi quyết định tự duyệt hoặc chuyển admin. Nhánh local thực hiện cùng nhóm quy tắc qua repository thay cho HTTP. Với chuỗi tuần, mọi buổi đều được kiểm xung đột trước khi chốt lưu.



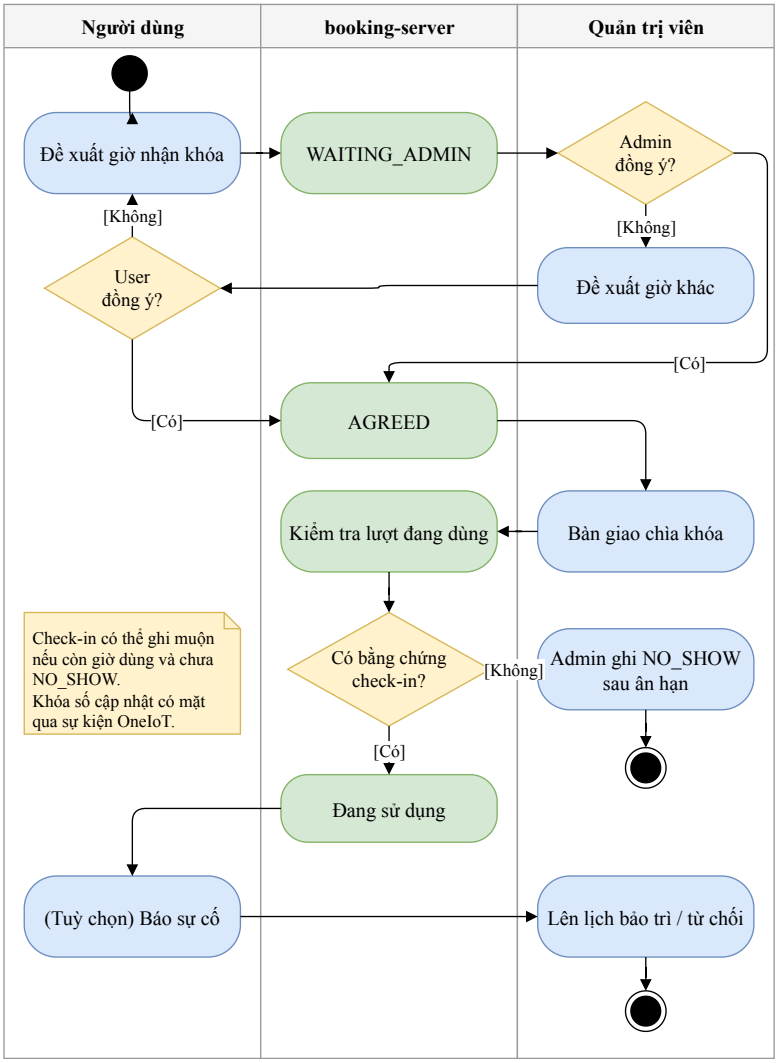
Hình 2.4: Luồng tạo yêu cầu và quyết định duyệt trong nhánh REST

Hình 2.5 tách sinh/lưu PIN và gửi lệnh tới khóa. PIN có bảy chữ số; thời hạn lấy từ booking và khoảng đệm cấu hình. Thuộc tính *lockDeliveredAt* lưu thời điểm broker nhận lệnh gửi mã. Luồng app–OneIoT–SmartLock đã được nhóm chạy thực tế để sử dụng mã và thao tác khóa.



Hình 2.5: Luồng tạo PIN, chờ gửi và giao tiếp OneIoT

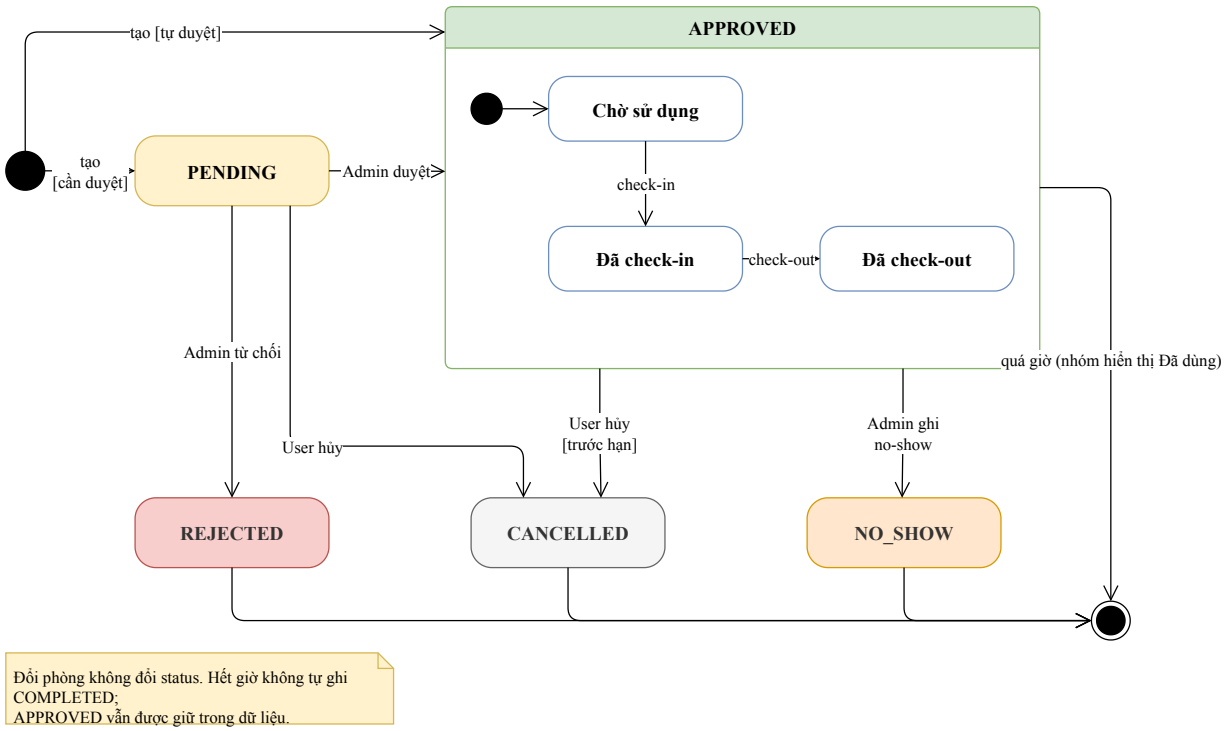
Hình 2.6 giữ riêng lịch nhận khóa, có mặt/vắng mặt và xử lý bảo trì. Lịch hẹn phải ở tương lai và trước giờ sử dụng; check-in thủ công chỉ được ghi trong giờ dùng. Có mặt muộn vẫn có thể được xác nhận nếu booking đang hợp lệ và chưa bị admin đóng bằng no-show.



Hình 2.6: Thỏa thuận nhận khóa, xác nhận có mặt và xử lý bảo trì

2.2.4 Vòng đời trạng thái đặt phòng

Trạng thái lưu là *PENDING*, *APPROVED*, *REJECTED*, *CANCELLED* và *NO_SHOW*. Tự duyệt cho phép tạo thẳng *APPROVED*. Check-in/check-out là các trường/sự kiện bên trong một lượt đã duyệt; đổi phòng không tự đổi trạng thái. Khi hết giờ, UI phân nhóm lượt đã dùng theo thời gian, không ghi thêm trạng thái *COMPLETED* vào model. Hình 2.7 thể hiện các trạng thái lưu và sự kiện sử dụng của lượt đặt.



Hình 2.7: Vòng đời booking và các mốc sử dụng liên quan

2.3 Yêu cầu chức năng

Bảng 2.2: Yêu cầu chức năng và minh chứng kiểm thử

ID / ưu tiên	Yêu cầu và luồng chính	Minh chứng hiện có
FR01 / Cao	Đăng ký, đăng nhập, đổi/khôi phục mật khẩu; đọc role từ tài khoản.	Ảnh Hình 4.1; TC01, TC17 và D01–D03.
FR02 / Cao	Tìm phòng theo tầng, giờ, sức chứa, thiết bị, loại khóa; trạng thái theo lịch.	Ảnh Hình 4.2; TC02, TC12.
FR03 / Cao	Tạo booking một lần/lặp; chặn trùng lịch và bảo trì; hủy/sửa chuỗi theo điều kiện.	TC02, TC11–TC12, TC21; kiểm thử quy tắc và vòng đời đặt phòng.
FR04 / Cao	Admin duyệt/từ chối/đổi phòng; tự duyệt phòng thường khi bật công tắc.	TC07, TC20, TC24; kiểm thử tự duyệt, duyệt tay và đổi phòng.
FR05 / Cao	Quyền PIN theo user–phòng, PIN 7 số và thời hạn; hẹn nhận chìa/ủy quyền.	TC03, TC10, TC13–TC15, TC19; thử nghiệm app–SmartLock.
FR06 / Trung bình	Ghi nhận sự kiện khóa, check-in thủ công khóa cơ và no-show có điều kiện.	TC10, TC22; kiểm thử sự kiện, check-in và ngưỡng no-show.
FR07 / Cao	Tách ghi chú với yêu cầu bảo trì; admin xử lý lịch/yêu cầu.	Ảnh Hình 4.3; TC02, TC23.
FR08 / Cao	Admin quản lý tài khoản/phòng/cấu hình; user sửa hồ sơ và xem thông báo.	Ảnh Hình 4.1; TC06, TC08–TC09, TC16, TC18.

2.4 Quy tắc nghiệp vụ và các trường hợp biên

Bảng 2.3 trình bày các mã BR để đặc tả và kiểm thử tham chiếu cùng một quy tắc. Giá trị trong bảng là mặc định của model cấu hình, không phải hằng số cố định cho mọi lần sử dụng.

Bảng 2.3: Quy tắc nghiệp vụ đối chiếu với mã nguồn

Mã	Điều kiện xử lý	Giá trị / phạm vi
BR-01	Phòng phải tồn tại và có trạng thái cho phép đặt; phương thức tiếp cận lấy từ loại khóa của phòng.	*AVAILABLE*.

Quy tắc nghiệp vụ (tiếp)

Mã	Điều kiện xử lý	Giá trị / phạm vi
BR-02	Ngày/giờ hợp lệ, bắt đầu trước kết thúc; giao nhau dùng khoảng nửa mở nên hai lượt liền kề không xung đột.	Cùng ngày, start trước end.
BR-03	Cửa sổ đặt trước áp dụng cho lượt đầu; khoảng dùng chưa kết thúc. Min bằng 0 cho phép cùng ngày đã bắt đầu nhưng còn thời gian.	Mặc định 1–3 ngày.
BR-04	Không chồng lịch bảo trì chưa hủy của đúng phòng.	Áp dụng từng buổi.
BR-05	Không trùng lượt còn hoạt động của cùng phòng hoặc cùng user; các trạng thái giữ chỗ là PENDING/APPROVED.	Kiểm theo khoảng giờ.
BR-06	Mục đích không trống; hạn mức kiểm ở lượt đầu khi tạo chuỗi, không cộng lại điều kiện hạn mức cho từng tuần tiếp theo.	Mặc định 2 lượt; chuỗi UI 1–8 tuần.
BR-07	Duyệt tay chỉ cho lượt PENDING chưa hết giờ; kiểm lại bảo trì và lịch đã duyệt. Tự duyệt chỉ sau validation, công tắc bật và phòng không đặc biệt.	Công tắc mặc định tắt.
BR-08	Chỉ chủ lượt hủy PENDING/APPROVED trước khi bắt đầu; lượt đã duyệt còn phải thỏa hạn hủy.	Mặc định 30 phút.
BR-09	Admin đổi lượt APPROVED trước hạn; phòng khác khả dụng, cùng loại khóa, đủ sức chứa/thiết bị, không xung đột/bảo trì; có lý do.	Mặc định 30 phút.
BR-10	PIN chỉ cho lượt đã duyệt ở phòng khóa số đang gắn SmartLock, chưa kết thúc và chưa có PIN chưa thu hồi.	7 số; đệm mặc định 10 phút.
BR-11	User tạo PIN phải là chủ lượt và có quyền active đúng phòng; admin có thể cấp/thu hồi quyền cặp user–phòng.	Không đổi vai trò tài khoản.
BR-12	Thỏa thuận/ủy quyền nhận khóa chỉ cho lượt đã duyệt dùng khóa cơ/thẻ; lịch nhận ở tương lai và trước giờ dùng.	Admin xác nhận địa điểm.
BR-13	Check-in thủ công cho khóa cơ/thẻ trong giờ dùng; no-show chỉ khi đã qua ân hạn và chưa có bằng chứng check-in.	Ân hạn mặc định 15 phút.
BR-14	Ghi chú hoặc yêu cầu bảo trì do chủ lượt APPROVED gửi trong giờ sử dụng; chỉ yêu cầu bảo trì có vòng đời xử lý riêng.	Ghi chú tối đa 500 ký tự.

Một số khác biệt giữa hai chế độ dữ liệu và phạm vi xử lý gồm:

- Đặt lặp là dãy cách nhau bảy ngày. UI giới hạn 1–8 tuần; khi bị gọi trực tiếp, service local chặn giá trị nhỏ hơn 1 nhưng kẹp giá trị trên 8 về 8, còn backend từ chối ngoài 1–8.
- Hủy, đổi phòng hoặc no-show cập nhật *revokedAt* để dừng sử dụng PIN của lượt đặt trong ứng dụng. Quyền room–user được quản lý riêng với PIN của từng booking.
- Local thực hiện read–validate–write trên Async Storage; REST kiểm quyền theo phiên và dùng transaction/khóa ở backend. Hai nhánh không có cơ chế đồng bộ hay hòa giải dữ liệu tự động.

2.5 Đặc tả các use case

Các mục 2.5.1–2.5.11 đặc tả các tác vụ tài khoản, đặt phòng, tiếp cận phòng và quản trị. Mục 2.5.12–2.5.13 mô tả điểm danh/no-show và ghi chú phòng. Mã FR liên kết với Bảng 2.2, mã BR liên kết với Bảng 2.3. Vị trí triển khai và kỹ thuật xử lý được tổng hợp tại Chương 4.

2.5.1 UC01 – Đăng nhập và các nhánh tài khoản

Trường	Nội dung đặc tả
Yêu cầu / quy tắc	FR01
Tác nhân	Người dùng và quản trị viên.
Tiền điều kiện	Tài khoản đăng nhập đã tồn tại và đang hoạt động; nhánh đăng ký tạo tài khoản user mới.
Luồng chính	1. Nhập tên đăng nhập và mật khẩu; hệ thống kiểm các ô bắt buộc và thực hiện xác thực. 2. Chuẩn hóa username, kiểm mật khẩu và active. Local đọc kho tài khoản; REST gọi API và nhận token/username/role. 3. Lưu phiên trong bộ nhớ ứng dụng và mở dashboard tương ứng vai trò; những request cần xác thực mang token của phiên. 4. Khi đăng xuất, xóa phiên/token và ngắt kết nối OneIoT theo vòng đời app.
Ngoại lệ / nhánh khác	Sai thông tin/tài khoản khóa: không tạo phiên. REST lỗi mạng/timeout: giữ form và báo lỗi, không tự chuyển local. Đăng ký kiểm username 3–30 ký tự theo tập cho phép, mật khẩu ít nhất 4 ký tự, hai mật khẩu khớp; họ tên và email là thông tin tùy chọn. Mã khôi phục là mã tĩnh của tài khoản, khác PIN mở khóa. Khi đăng ký từ app, bỏ trống dùng *000000* ở cả local/REST; mã tự nhập có ít nhất 4 ký tự. Đổi mật khẩu kiểm mật khẩu hiện tại; khôi phục kiểm username/mã khôi phục.
Hậu điều kiện	Đăng nhập thành công xác định đúng role; lỗi không cấp quyền vào dashboard. Khi khởi động lại ứng dụng, người dùng đăng nhập để tạo phiên mới.

2.5.2 UC02 – Tìm và đặt phòng một lần hoặc lặp tuần

Trường	Nội dung đặc tả
Yêu cầu / quy tắc	FR02, FR03; BR-01–BR-07
Tác nhân	User đã được xác thực.
Tiền điều kiện	Tài khoản còn hoạt động; có phòng và cấu hình nghiệp vụ; nguồn dữ liệu local hoặc REST được chọn trước.
Luồng chính	1. Mở Tìm và đặt phòng, chọn ngày/khung giờ và tầng; có thể lọc sức chứa, thiết bị và loại khóa. 2. Xem các phòng của tầng cùng trạng thái tính từ phòng, khoảng giờ và lịch bảo trì/booking; chọn phòng đang khả dụng. 3. Nhập mục đích; nếu đặt lặp, chọn số tuần. Hệ thống tạo từng ngày cách nhau bảy ngày và gán cùng mã chuỗi. 4. Kiểm phòng, ngày/giờ, xung đột phòng/user, bảo trì, mục đích và hạn mức; kiểm đủ mọi buổi trước khi chốt ghi chuỗi. 5. Nếu chính sách tự duyệt áp dụng cho phòng thường, lưu APPROVED, cấp quyền PIN khi phù hợp và gửi thông báo; trường hợp còn lại lưu PENDING và báo admin. 6. Làm mới sơ đồ và hiển thị kết quả tạo yêu cầu.
Ngoại lệ / nhánh khác	Phòng không còn khả dụng hoặc vi phạm một quy tắc: báo lý do, không tạo yêu cầu/chuỗi dở dang. Min đặt trước bằng 0 cho phép cùng ngày nếu chưa hết giờ. Số tuần UI là 1–8; khác biệt validation local/server được ghi sau Bảng 2.3. Cửa sổ và hạn mức chỉ kiểm như lượt đầu; xung đột/bảo trì vẫn kiểm mọi tuần.
Hậu điều kiện	Một hoặc nhiều booking được lưu theo kết quả tự duyệt/PENDING; các khoảng PENDING/APPROVED giữ chỗ. Cách duyệt được quyết định bởi cấu hình tự duyệt và loại phòng.

2.5.3 UC03 – Duyệt hoặc từ chối yêu cầu

Trường	Nội dung đặc tả
Yêu cầu / quy tắc	FR04; BR-07, BR-11
Tác nhân	Admin; hệ thống thực hiện nhánh tự duyệt theo cấu hình.
Tiền điều kiện	Duyệt tay yêu cầu admin còn quyền và booking PENDING; duyệt chuỗi chỉ xét các lượt còn chờ trong cùng chuỗi.
Luồng chính	1. Mở yêu cầu chờ, đọc người đặt, phòng, thời gian, mục đích và thông tin chuỗi nếu có. 2. Chọn duyệt hoặc từ chối. Khi duyệt, kiểm lại chưa hết giờ, không trùng bảo trì hoặc booking đã duyệt khác. 3. Ghi trạng thái, người và thời điểm xử lý; với phòng PIN, cấp quyền tạo mã đúng cặp user–phòng. 4. Gửi thông báo cho người đặt và cập nhật danh sách. Với duyệt chuỗi, kiểm các ứng viên trước khi chốt cả nhóm.
Ngoại lệ / nhánh khác	Yêu cầu đã xử lý: từ chối thao tác lặp. Vi phạm điều kiện duyệt: giữ trạng thái, nêu nguyên nhân. Từ chối không cấp quyền PIN. Tự duyệt xảy ra khi tạo sau validation, chỉ nếu công tắc bật và phòng không yêu cầu duyệt riêng.
Hậu điều kiện	Booking hoặc chuỗi được xử lý nhất quán trong phạm vi nguồn dữ liệu. Quyền PIN theo phòng không phải quyền quản trị tài khoản; mã PIN chưa được tạo chỉ bởi thao tác duyệt.

2.5.4 UC04 – Đổi phòng thay thế

Trường	Nội dung đặc tả
Yêu cầu / quy tắc	FR04; BR-09
Tác nhân	Admin.
Tiền điều kiện	Booking APPROVED; còn đủ thời gian trước giờ bắt đầu theo cấu hình; có phòng thay thế phù hợp.
Luồng chính	1. Chọn booking cần đổi; xem các phòng khác khả dụng cùng loại khóa, sức chứa không thấp hơn và có đủ thiết bị của phòng cũ. 2. Chọn phòng thay thế và nhập lý do. 3. Kiểm hạn đổi, tình trạng phòng, lịch bảo trì và các booking PENDING/APPROVED chồng giờ của phòng mới. 4. Cập nhật roomId và lịch sử đổi; bỏ lịch hẹn/thỏa thuận nhận khóa cũ, đánh dấu PIN cũ thu hồi trong dữ liệu. 5. Nếu phòng mới là PIN, cấp quyền cho phòng mới; thông báo cho chủ booking.
Ngoại lệ / nhánh khác	Không cho chọn lại phòng cũ, phòng khác loại khóa, thiếu sức chứa/thiết bị, phòng bận/bảo trì, thiếu lý do hoặc quá hạn. Mặc định phải còn ít nhất 30 phút; nếu admin đổi tham số, dùng giá trị mới.
Hậu điều kiện	Lượt giữ trạng thái APPROVED, trỏ phòng mới và có lịch sử đổi. PIN cũ được đánh dấu thu hồi; quyền theo phòng được quản lý riêng.

2.5.5 UC05 – Tạo và gửi PIN tạm thời

Trường	Nội dung đặc tả
Yêu cầu / quy tắc	FR05; BR-10, BR-11
Tác nhân	Chủ booking hoặc admin; OneIoT/khóa tham gia đường gửi và phản hồi sự kiện.
Tiền điều kiện	Booking APPROVED ở phòng PIN đang gắn SmartLock, chưa kết thúc. User phải là chủ lượt và có quyền active tại đúng phòng; admin có thể tạo hộ.
Luồng chính	1. Mở booking và yêu cầu tạo PIN; kiểm role, sở hữu, trạng thái, loại khóa và quyền phòng. 2. Từ chối nếu booking đã có một PIN chưa được đánh dấu thu hồi; sinh mã bảy chữ số cho lượt hợp lệ. 3. Tính validFrom/validUntil theo giờ booking và khoảng đệm; lấy passwordId của khóa đang gắn phòng rồi lưu PIN. 4. Nếu có phiên OneIoT, phát lệnh qua native MQTT; nếu chưa có, hiển thị mã ở trạng thái chờ gửi. 5. Khi nối lại OneIoT, xét các mã chờ đúng khóa/phòng và chưa hết hạn; lưu kết quả publish hoặc lỗi gửi.
Ngoại lệ / nhánh khác	Khác chủ lượt, quyền bị thu hồi, phòng không gắn khóa, đã hết giờ hoặc có PIN chưa thu hồi: báo lỗi. Lỗi gửi không làm mất PIN đã lưu; mã hết hiệu lực/đã thu hồi không được gửi lại. passwordId được cấp theo dải 1–255 và quay vòng.
Hậu điều kiện	PIN và metadata giao lệnh được lưu trong booking; lockDeliveredAt ghi thời điểm broker nhận publish. Hủy/đổi/no-show cập nhật trạng thái PIN của lượt đặt.

2.5.6 UC06 – Hủy lượt đặt hoặc các lượt tương lai

Trường	Nội dung đặc tả
Yêu cầu / quy tắc	FR03; BR-08, BR-10
Tác nhân	Chủ booking.
Tiền điều kiện	Tài khoản user hoạt động; booking thuộc người gọi, PENDING hoặc APPROVED và chưa bắt đầu.
Luồng chính	1. Mở Đặt phòng của tôi, chọn hủy một lượt hoặc các lượt tương lai trong chuỗi. 2. Kiểm sở hữu và trạng thái; lượt đã duyệt phải còn đủ hạn hủy theo cấu hình. 3. Chuyển các lượt đủ điều kiện sang CANCELLED và đánh dấu revokedAt cho PIN đã có. 4. Lưu dữ liệu, gửi thông báo cho admin và làm mới danh sách/lịch phòng.
Ngoại lệ / nhánh khác	Lượt đã bắt đầu, không thuộc người gọi, đã từ chối/hủy/no-show hoặc lượt APPROVED quá hạn: báo lỗi. Hủy chuỗi chỉ tác động các buổi tương lai đủ điều kiện; lịch sử và các buổi đã sử dụng được giữ nguyên.
Hậu điều kiện	Các khoảng đã hủy không còn giữ chỗ; lịch sử vẫn được giữ. PIN của lượt đặt được đánh dấu thu hồi; quyền theo phòng được quản lý riêng.

2.5.7 UC07 – Thỏa thuận nhận khóa và ủy quyền nhận hộ

Trường	Nội dung đặc tả
Yêu cầu / quy tắc	FR05; BR-12
Tác nhân	Chủ booking và admin.
Tiền điều kiện	Booking APPROVED cho phòng *PHYSICAL_KEY*; thời điểm nhận khóa phải ở tương lai và trước giờ dùng.
Luồng chính	1. User đề xuất ngày/giờ nhận; trạng thái thỏa thuận chuyển *WAITING_ADMIN*. 2. Admin chấp nhận kèm địa điểm hoặc gửi đề xuất khác; đề xuất khác chuyển *WAITING_USER*. 3. Bên còn lại chấp nhận hoặc đề xuất lại; hệ thống kiểm thời gian và người có quyền trước mỗi lần phản hồi. 4. Khi thống nhất, lưu lịch hẹn và lịch sử đề xuất ở trạng thái AGREED; gửi thông báo cho bên còn lại. 5. Nếu cần nhận hộ, chủ lượt khai báo họ tên và mã định danh người được ủy quyền trước khi giờ sử dụng bắt đầu.
Ngoại lệ / nhánh khác	Phòng PIN, khác chủ lượt, thiếu họ tên/mã nhận hộ, thiếu địa điểm khi xác nhận, đề xuất đã quá giờ hoặc đã thống nhất: chặn theo điều kiện tương ứng. User đang chờ admin không gửi chồng đề xuất; bên đề xuất không tự xác nhận thay bên kia. Admin còn có đường đặt lịch trực tiếp qua scheduleKeyPickup.
Hậu điều kiện	Ứng dụng có lịch hẹn/người nhận hộ và lịch sử trao đổi. Nhân viên quản lý đối chiếu giấy tờ và thực hiện giao, nhận chìa theo lịch đã thống nhất.

2.5.8 UC08 – Yêu cầu và quản lý lịch bảo trì

Trường	Nội dung đặc tả
Yêu cầu / quy tắc	FR07; BR-04, BR-14
Tác nhân	User đang sử dụng phòng; admin xử lý yêu cầu và lịch.
Tiền điều kiện	User gửi yêu cầu cho booking của mình đã duyệt, đang trong giờ dùng; admin có quyền quản lý phòng/bảo trì.
Luồng chính	1. User nhập lý do; hệ thống kiểm chủ lượt, thời gian và không có yêu cầu bảo trì PENDING khác của cùng booking. 2. Lưu yêu cầu PENDING và thông báo admin; yêu cầu này tự nó chưa là lịch bảo trì. 3. Admin tiếp nhận bằng cách chọn đúng phòng, ngày/giờ, lý do, hoặc từ chối kèm ghi chú. 4. Khi lập lịch, kiểm phòng tồn tại, khoảng giờ, trùng bảo trì và booking APPROVED; lưu lịch và liên kết yêu cầu đã xử lý. 5. Các lần đặt/duyệt sau kiểm lịch bảo trì; admin có thể hủy lịch, giữ lại dấu thời điểm hủy.
Ngoại lệ / nhánh khác	Sai chủ lượt/ngoài giờ dùng, lý do rỗng, yêu cầu trùng hoặc đã xử lý: báo lỗi. Lịch trùng booking đã duyệt phải được giải quyết trước khi lập bảo trì. Tiếp nhận không được đổi lịch sang phòng khác với phòng bị báo. Admin có thể lập lịch trực tiếp không qua yêu cầu user.
Hậu điều kiện	Yêu cầu có kết quả SCHEDULED/REJECTED; lịch bảo trì chưa hủy chặn khung giờ theo BR-04. Ghi chú phòng tại UC13 không tạo yêu cầu hay lịch bảo trì.

2.5.9 UC09 – Cấu hình quy tắc nghiệp vụ

Trường	Nội dung đặc tả
Yêu cầu / quy tắc	FR04, FR08; BR-03, BR-06–BR-10, BR-13
Tác nhân	Admin.
Tiền điều kiện	Phiên/tài khoản có quyền admin; đã đọc cấu hình hiện có.
Luồng chính	1. Mở màn hình quy định, xem cửa sổ đặt trước, hạn mức, hạn hủy/đổi, đệm PIN, ân hạn no-show, thông báo và tự duyệt. 2. Thay đổi các tham số cần thiết; hệ thống kiểm số nguyên không âm, min không lớn hơn max và hạn mức ít nhất một. 3. Lưu cấu hình; các service lấy giá trị mới cho những lần xử lý tiếp theo. 4. Khi bật tự duyệt, phòng thường có thể được duyệt ngay lúc tạo; phòng requiresApproval vẫn chờ admin.
Ngoại lệ / nhánh khác	User không được lưu cấu hình; số âm/không nguyên, min lớn hơn max hoặc hạn mức bằng 0 bị từ chối. Backend giữ trường mới đang có khi client cũ cập nhật mà bỏ qua trường đó, tránh vô tình tắt tự duyệt.
Hậu điều kiện	Cấu hình được lưu tại nguồn local hoặc REST đang chọn. Thay đổi cấu hình không tự hòa giải hai kho dữ liệu, không sửa ngược mọi PIN hoặc lịch hẹn đã được tạo.

2.5.10 UC10 – Quản lý liên kết SmartLock và phiên OneIoT

Trường	Nội dung đặc tả
Yêu cầu / quy tắc	FR05, FR06, FR08; BR-10, BR-11
Tác nhân	Admin quản lý liên kết; user có quyền sử dụng chức năng kết nối/gửi mã của lượt mình.
Tiền điều kiện	Có model khóa đang quản lý; gắn khóa yêu cầu phòng PIN. Kết nối thật cần Internet, token và thông tin thiết bị phù hợp.
Luồng chính	1. Admin chọn một phòng khóa số và lưu liên kết với SmartLock đang quản lý; muốn đổi phòng thì cập nhật liên kết. 2. Người dùng chức năng được phép nhập token cho phiên và kiểm tra kết nối OneIoT. 3. Khi kết nối thành công, app đăng ký nhận bản tin và xét gửi các PIN chờ của đúng phòng/khóa. 4. UI hiển thị kết nối/lỗi; token ở RAM và kết nối được dọn khi logout hoặc app xuống nền. 5. Admin có thể tháo liên kết; các PIN/phòng không còn khớp liên kết không được gửi như mã của khóa hiện tại.
Ngoại lệ / nhánh khác	Phòng không tồn tại hoặc khóa cơ/thẻ: không cho gắn. Token/mạng sai: hiển thị lỗi kết nối và giữ mã ở trạng thái chờ gửi. Một khóa được quản lý và gắn với tối đa một phòng tại một thời điểm.
Hậu điều kiện	Lưu quan hệ phòng–khóa và metadata gửi/nhận theo phiên. Ứng dụng gửi mã, nhận sự kiện và cập nhật trạng thái khi phiên OneIoT hoạt động.

2.5.11 UC11 – Quản lý tài khoản và quyền PIN theo phòng

Trường	Nội dung đặc tả
Yêu cầu / quy tắc	FR01, FR05, FR08; BR-11
Tác nhân	Admin.
Tiền điều kiện	Phiên/tài khoản có quyền admin; dữ liệu tài khoản và quyền được đọc từ nguồn đang chọn.
Luồng chính	1. Tạo tài khoản quản lý bằng username, mật khẩu, mã khôi phục và role; kiểm tên hợp lệ/chưa tồn tại, mật khẩu/mã khôi phục ít nhất bốn ký tự. 2. Tìm tài khoản; thay đổi role, trạng thái active, mật khẩu hoặc mã khôi phục theo chức năng được cung cấp. 3. Khi xóa, kiểm không phải tài khoản đang đăng nhập và không còn booking trạng thái PENDING/APPROVED cần xử lý. 4. Chọn user và phòng PIN để cấp/thu hồi quyền tạo mã; lưu người/thời điểm xử lý và thông báo. 5. Mỗi lần user tạo PIN, kiểm lại chủ booking, trạng thái và quyền phòng; việc cấp quyền không thay role.
Ngoại lệ / nhánh khác	Chặn trùng/tên sai, mật khẩu/mã quản lý quá ngắn; chặn tự khóa, tự hạ quyền hoặc tự xóa tài khoản admin đang dùng. Thu hồi quyền yêu cầu bản ghi active. Thu hồi quyền phòng ngăn việc tạo PIN tiếp theo; vòng đời PIN đã cấp được quản lý riêng theo lượt đặt.
Hậu điều kiện	Tài khoản bị khóa không đăng nhập được; quyền PIN phản ánh đúng cặp user–phòng. Điều kiện xóa tài khoản được xác định theo trạng thái booking liên quan.

2.5.12 UC12 – Ghi nhận có mặt, sự kiện khóa và vắng mặt

Trường	Nội dung đặc tả
Yêu cầu / quy tắc	FR06; BR-13
Tác nhân	Admin đối với check-in thủ công/no-show; SmartLock/OneIoT đối với bản tin sự kiện.
Tiền điều kiện	Booking APPROVED; check-in thủ công chỉ cho phòng khóa cơ/thẻ trong giờ dùng. No-show cần hết ân hạn và chưa có bằng chứng check-in.
Luồng chính	1. Với khóa cơ/thẻ, admin chọn xác nhận có mặt; service kiểm đúng loại phòng, khoảng dùng và chưa có bằng chứng check-in. 2. Với khóa số, app phân tích sự kiện MQTT, đối chiếu khóa/phòng/thời gian và cập nhật trường/sự kiện của booking phù hợp. 3. Nếu qua ân hạn và chưa có checkedInAt, checkedOutAt hoặc sự kiện *CHECK_IN*, admin có thể chọn ghi nhận vắng mặt. 4. Lưu người/thời điểm xác nhận; no-show chuyển *NO_SHOW* và đánh dấu PIN thu hồi trong dữ liệu; thông báo người đặt.
Ngoại lệ / nhánh khác	Không cho check-in thủ công sai loại khóa, ngoài giờ dùng hoặc đã có bằng chứng. Không ghi no-show trước ân hạn hoặc sau khi có bằng chứng có mặt. Quá ân hạn không tự động chạy tác vụ no-show. Việc ghi nhận vắng mặt căn cứ vào dữ liệu booking và thao tác admin; nhận sự kiện khóa yêu cầu phiên MQTT đang hoạt động.
Hậu điều kiện	Dấu có mặt/ra được lưu riêng với trạng thái APPROVED; *NO_SHOW* đóng lượt và giải phóng giữ chỗ. Sự kiện SmartLock được xử lý để cập nhật trạng thái có mặt/ra của lượt sử dụng.

2.5.13 UC13 – Ghi chú tình trạng phòng đang sử dụng

Trường	Nội dung đặc tả
Yêu cầu / quy tắc	FR07; BR-14
Tác nhân	Chủ booking đang sử dụng; admin nhận thông tin.
Tiền điều kiện	Booking của user, APPROVED và thời điểm hiện tại trong khoảng dùng.
Luồng chính	1. User nhập ghi chú về đồ dùng hoặc điều cần lưu ý trong lượt đang dùng. 2. Kiểm nội dung sau trim không rỗng, không quá 500 ký tự; kiểm chủ lượt và thời gian. 3. Gửi thông báo chú ý cho admin kèm phòng, lịch và nội dung; UI báo kết quả/lỗi. 4. Admin đọc thông báo và quyết định xử lý nghiệp vụ tiếp theo nếu cần.
Ngoại lệ / nhánh khác	Ngoài giờ dùng, khác chủ lượt, trạng thái không hợp lệ hoặc nội dung quá dài/rỗng: từ chối. Lỗi REST không được hiển thị như gửi thành công. Banner chỉ hiện tức thời nếu người nhận là tài khoản đang đăng nhập; thông báo của admin không hiện cho user.
Hậu điều kiện	Ghi chú được đưa vào thông báo cho admin; không tạo MaintenanceRequest hay lịch chặn phòng. Nếu cần luồng bảo trì có theo dõi trạng thái, dùng UC08.

2.6 Yêu cầu phi chức năng

Bảng 2.4: Tiêu chí phi chức năng và giới hạn đánh giá

ID / nhóm	Tiêu chí và điều kiện	Bằng chứng / giới hạn
NFR01 / Khả dụng	Tìm và gửi yêu cầu cùng màn hình; nút/lỗi theo vai trò; loading khi xử lý.	Ảnh giao diện, kiểm thử component và thao tác trên Android xác nhận các bước sử dụng.
NFR02 / Phản hồi	HTTP có timeout 12 giây và thông báo mất mạng/JSON lỗi; tránh chờ vô hạn.	Kiểm thử API xác nhận timeout và phản hồi lỗi; phạm vi đánh giá định lượng trình bày ở mục 5.3.
NFR03 / Quyền và dữ liệu	User thao tác đúng lượt/phòng; token API trong RAM; token OneIoT theo phiên.	Test phân quyền và H2; mật khẩu/PIN còn dạng rõ, xác minh chứng thư native TLS là hướng gia cố bảo mật khi mở rộng.
NFR04 / Tái lập	Mã nguồn, lockfile, cấu hình và hướng dẫn chạy; hồ sơ ghi rõ điều kiện kiểm thử.	README, package-lock, test và biên bản. Server public và khóa thật phụ thuộc môi trường ngoài.

2.7 Tiêu chí chấp nhận và mức độ ưu tiên

FR01–FR08 bao phủ các luồng nghiệp vụ của sản phẩm, từ đăng nhập, đặt/duyệt phòng đến tiếp cận bằng khóa, điểm danh và quản trị. Nhóm đã hoàn thành kiểm thử chức năng của ứng dụng và SmartLock trong môi trường thực tế; các ca kiểm thử và kết quả được trình bày ở Chương 5.

Bảng 2.5 cụ thể hóa điều kiện chấp nhận theo nhóm chức năng. Các yêu cầu mức Cao là chức năng bắt buộc; FR06 là nhóm hỗ trợ theo dõi sử dụng và xử lý vắng mặt, được triển khai cùng các luồng chính. Mở rộng nhiều tòa nhà, nhiều khóa và đồng bộ ngoại tuyến thuộc hướng phát triển.

Bảng 2.5: Điều kiện chấp nhận chức năng

Yêu cầu	Điều kiện chấp nhận	Ca kiểm chứng
FR01	Đăng nhập đúng vai trò; chặn sai mật khẩu/tài khoản khóa; đăng ký, đổi và khôi phục mật khẩu kiểm dữ liệu hợp lệ.	TC01, TC17; D01–D03.
FR02–FR03	Lọc đúng tiêu chí; tạo/hủy/sửa lượt hoặc chuỗi đúng quy tắc; từ chối lịch trùng, bảo trì và vượt hạn mức.	TC02, TC11–TC12, TC21.
FR04	Duyệt tay đúng quyền; tự duyệt chỉ áp dụng phòng thường khi bật cấu hình; phòng đặc biệt vẫn chờ admin.	TC07, TC20, TC24.
FR05	PIN có 7 chữ số, đúng chủ lượt/quyền phòng/thời hạn; lịch nhận chìa và ủy quyền đúng trình tự.	TC03, TC10, TC13–TC15, TC19.
FR06	Sự kiện có mặt/ra cập nhật lượt phù hợp; no-show chỉ sau ân hạn và khi chưa có check-in.	TC10, TC22.
FR07	Ghi chú chỉ tạo thông báo; yêu cầu bảo trì có trạng thái xử lý; lịch bảo trì hiệu lực chặn đặt phòng.	TC02, TC23.
FR08	Chặn user thực hiện quản trị; dữ liệu phòng/hồ sơ/cấu hình được lưu đúng; thông báo đúng người nhận.	TC06, TC08–TC09, TC16, TC18; D04–D06.

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG

3.1 Lựa chọn công nghệ và lập luận

Bảng 3.1 tổng hợp công nghệ, lý do lựa chọn và các hạn chế kỹ thuật cần quản lý khi phát triển hệ thống.

Bảng 3.1: Công nghệ, lý do lựa chọn và hạn chế kỹ thuật

Thành phần / phiên bản	Lý do lựa chọn	Hạn chế kỹ thuật
React Native 0.86.2; React 19.2.3 [4]	Giao diện component, tái sử dụng logic TypeScript và tích hợp Android native.	Tập trung vào Android; mở rộng nền tảng cần cấu hình native tương ứng.
TypeScript 5.9.3; Community CLI 20.1.0	Kiểm kiểu model/props; kiểm soát dự án native và bridge MQTT/thông báo.	Cần quản lý Android SDK/JDK, phụ thuộc native và cấu hình build.
useState/useEffect; điều hướng bằng state	Ghép các màn hình theo vai trò, truyền dữ liệu và callback trực tiếp.	Điều hướng bằng state yêu cầu xử lý BackHandler và cleanup chủ động.
Async Storage 3.1.1; fetch/Promise [7]	Kho JSON local gọn và REST client thống nhất.	Hai kho local/REST độc lập; đồng bộ ngoại tuyến cần cơ chế hòa giải và bảo vệ dữ liệu.
Java 17; Spring Boot 3.4.4; JPA/H2	REST, phân quyền, giao dịch và khóa dữ liệu; H2 thuận tiện phát triển/test.	BookingService tập trung nhiều nghiệp vụ; cần phân tách khi quy mô xử lý tăng.
Kotlin 2.1.20; MQTT/oneM2M	Native Android gửi lệnh khóa, nhận sự kiện và tích hợp thông báo nền tảng.	Kết nối MQTT gắn với phiên ứng dụng và cần mạng để trao đổi lệnh/sự kiện khóa.
Jest 29; react-test-renderer 19.2.3 [8]	Kiểm hàm, component và hợp đồng tích hợp bằng mocks có kiểm soát.	Mock giúp kiểm tra logic có kiểm soát; thử trên thiết bị bổ sung kiểm tra tích hợp.

3.1.1 React Native, TypeScript và trạng thái giao diện

Phiên đăng nhập được quản lý ở thành phần gốc; callback xác thực cập nhật trạng thái để chuyển từ biểu mẫu sang màn hình chức năng theo vai trò. Mỗi màn giữ input/loading/error; TypeScript kiểm kiểu model, props và hợp đồng với repository [5, 6]. Community CLI cho phép tích hợp MQTT/thông báo native Android. Điều hướng và trạng thái nghiệp vụ dùng state, props và service; BackHandler, tham số, làm mới và cleanup do app xử lý trực tiếp. Khi số màn tăng, cách này cần được đánh giá lại.

3.1.2 Java, Spring Boot và ánh xạ dữ liệu

Java 17 dùng record cho request/response; Spring truyền dependency qua constructor. Controller nhận HTTP và gọi service; *SessionAuthFilter* kế thừa *OncePerRequestFilter*; *ApiExceptionHandler* ánh xạ ngoại lệ sang HTTP [9]. JPA/Hibernate quản lý entity/giao dịch, converter triển khai *AttributeConverter* cho dữ liệu JSON lồng. Entity lưu trạng thái nghiệp vụ có thể cập nhật, còn record biểu diễn dữ liệu trao đổi bất biến. H2 mặc định lưu tệp, test dùng RAM; đổi hệ quản trị vẫn cần kiểm lược đồ, truy vấn và khóa [10].

3.1.3 MQTT, oneM2M và định dạng mã khóa

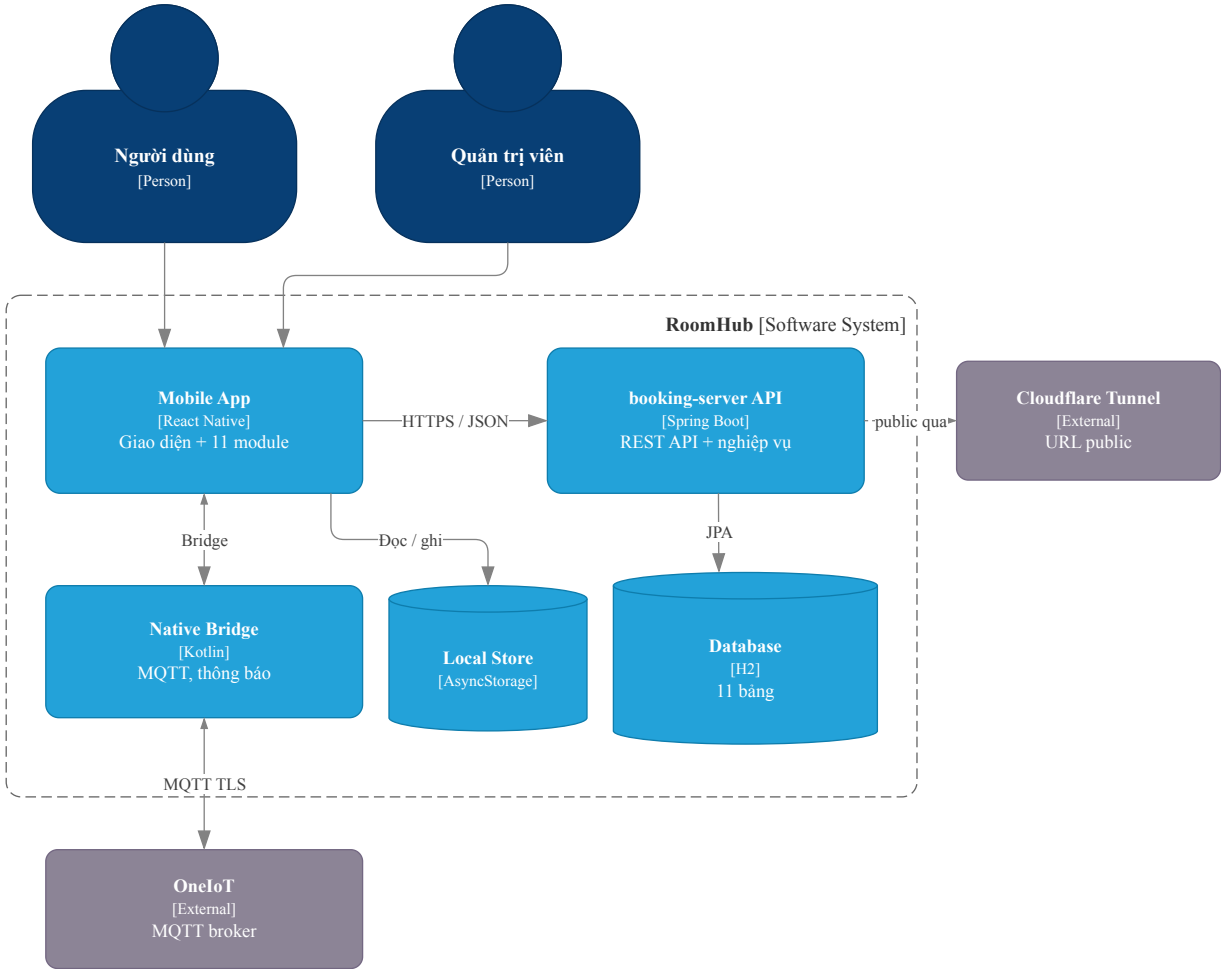
App gửi bản tin qua module Kotlin tới broker OneIoT, theo chủ đề và cấu trúc oneM2M [11, 12]. Native dùng AES/CTR/NoPadding để đóng gói mật khẩu theo định dạng thiết bị; source dẫn xuất khóa bằng SHA-256 và giá trị khởi tạo từ thời gian/số hiệu mật khẩu. Cách đóng gói này đáp ứng định dạng của thiết bị. PIN trong dữ liệu nghiệp vụ và cấu hình xác minh chứng thư TLS native là các điểm cần gia cố về bảo mật khi mở rộng vận hành.

MQTT PUBACK xác nhận broker nhận lệnh publish. App ghi trạng thái tạo/lưu/gửi mã và xử lý sự kiện khóa qua các bước riêng; chạy thử thực tế đã xác nhận luồng kết nối ứng dụng với SmartLock. Chi tiết được phân tích ở mục 3.6.2.

3.2 Kiến trúc và luồng dữ liệu

3.2.1 Ranh giới thành phần và hai nguồn dữ liệu

Hình 3.1 thể hiện các thành phần chạy độc lập. Ứng dụng Android gồm giao diện React Native và native bridge; backend cung cấp REST và lưu H2; OneIoT là kết nối riêng cho khóa. App giữ phiên ở cấp gốc, còn repository chọn local hoặc REST bằng cấu hình runtimeFlags.



Hình 3.1: Các thành phần chạy: app Android, REST backend và OneIoT

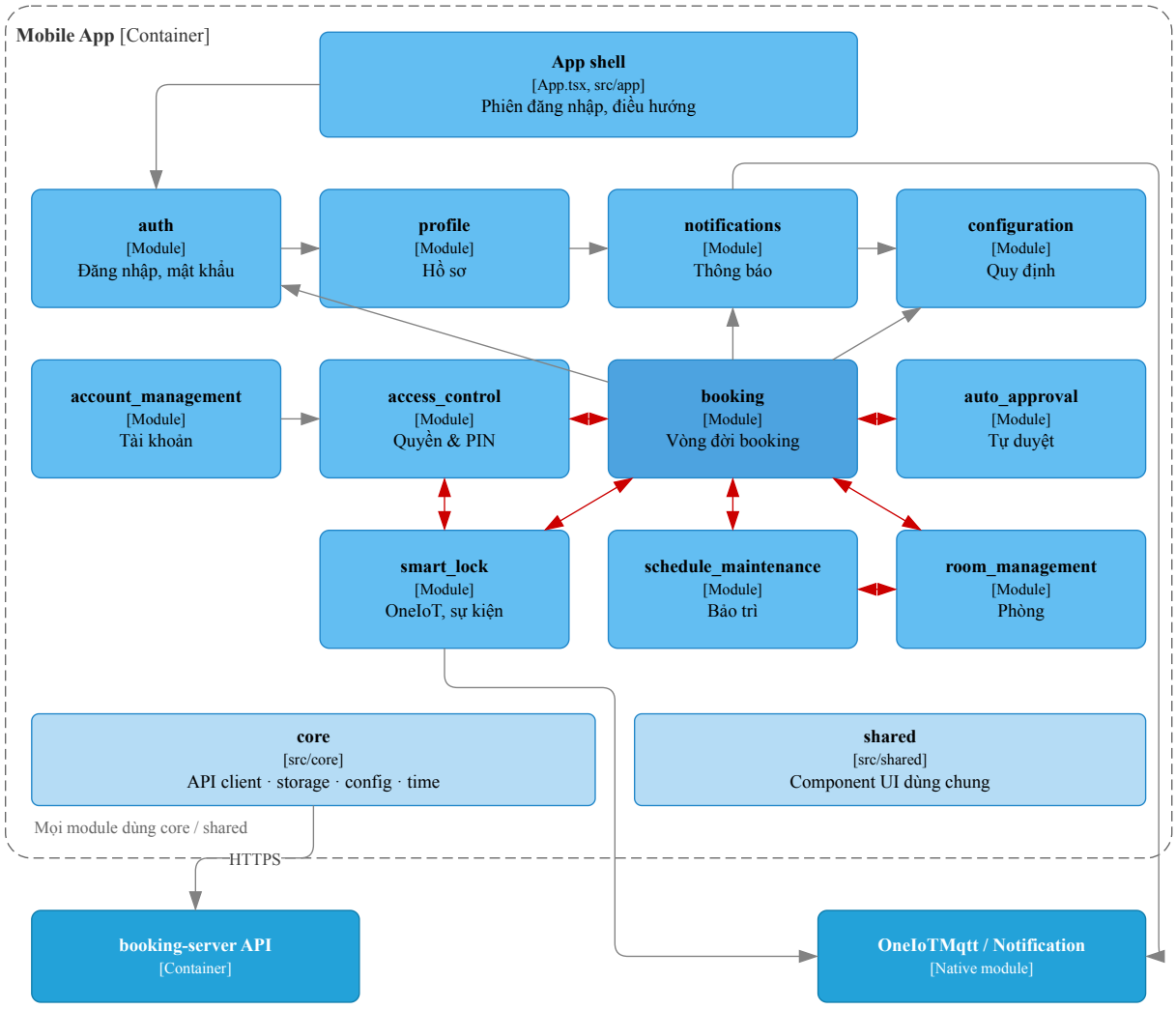
Luồng local là UI → repository → readJson/writeJson → Async Storage. Luồng remote là UI → apiRequest → SessionAuthFilter → controller theo nghiệp vụ → BookingService → JPA/H2. Luồng khóa là service TypeScript → module Kotlin → OneIoT theo MQTT/oneM2M. Công tắc

REST không bật hoặc tắt kết nối MQTT.

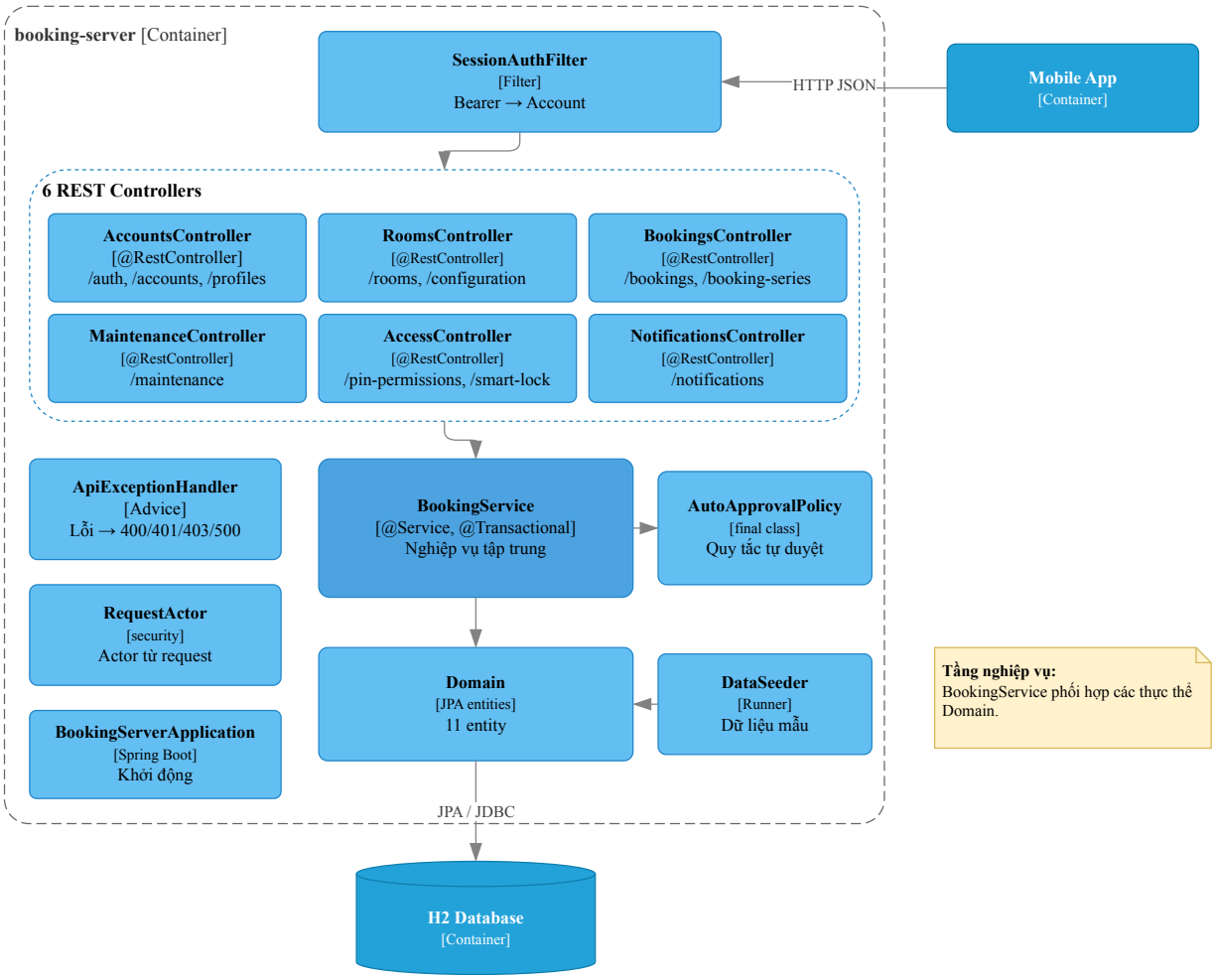
Nguồn local và nguồn REST dùng cùng giao diện nghiệp vụ ở màn hình nhưng là hai kho độc lập. Chế độ REST không tự lấy backlog local, không tự hòa giải dữ liệu nhiều máy và không tự chuyển về local khi mạng lỗi. Cấu hình bản build quyết định nguồn dữ liệu sử dụng.

3.2.2 Module ứng dụng và trách nhiệm lớp phía server

Hình 3.2 bổ sung cách các module cộng tác; Hình 3.3 cho biết ranh giới HTTP, xác thực, nghiệp vụ và dữ liệu ở server. Hai góc nhìn làm rõ phụ thuộc giữa các module trong app và đường xử lý yêu cầu phía Java.



Hình 3.2: Trách nhiệm và các phụ thuộc chính giữa module React Native



Hình 3.3: Các lớp giao tiếp, nghiệp vụ và lưu trữ của backend

Bảng 3.2: Trách nhiệm của 11 module nghiệp vụ

Module	Trách nhiệm và cộng tác
auth	Đăng ký/đăng nhập, mật khẩu và xác thực vai trò; tài khoản quản trị cũng dùng accountRepository.
account_management	UI quản lý tài khoản và quyền PIN; gọi auth và access control.
room_management	Danh mục, bộ lọc và sơ đồ tầng; màn tìm và đặt gọi nghiệp vụ booking. Quản lý phòng ghép với bảo trì.
booking	Tạo, duyệt, lịch lặp, hủy/đổi phòng, nhận chìa, ủy quyền, ghi chú và no-show; dùng phòng, cấu hình, bảo trì, tự duyệt và thông báo.
auto_approval	Tính quyết định duyệt từ cấu hình và dấu phòng đặc biệt sau khi yêu cầu đã qua validation.
access_control	Quyền theo user–phòng; tạo và lưu PIN theo booking đã duyệt, gọi smart lock khi có thể gửi.
smart_lock	Quản lý một khóa gắn phòng; cấp password ID, kết nối native OneIoT và tiếp nhận sự kiện.
schedule_maintenance	Yêu cầu bảo trì, lịch đã được admin lên và kiểm xung đột.
configuration	Cửa sổ đặt trước, hạn mức, các thời hạn và công tắc tự duyệt/thông báo.
profile	Hồ sơ người dùng theo tài khoản.
notifications	Lưu/đọc theo người nhận, banner và cầu thông báo native; chỉ hiện banner cho tài khoản đang đăng nhập.

Lớp hạ tầng dùng chung cung cấp HTTP client, lưu trữ JSON, cấu hình và xử lý thời gian;

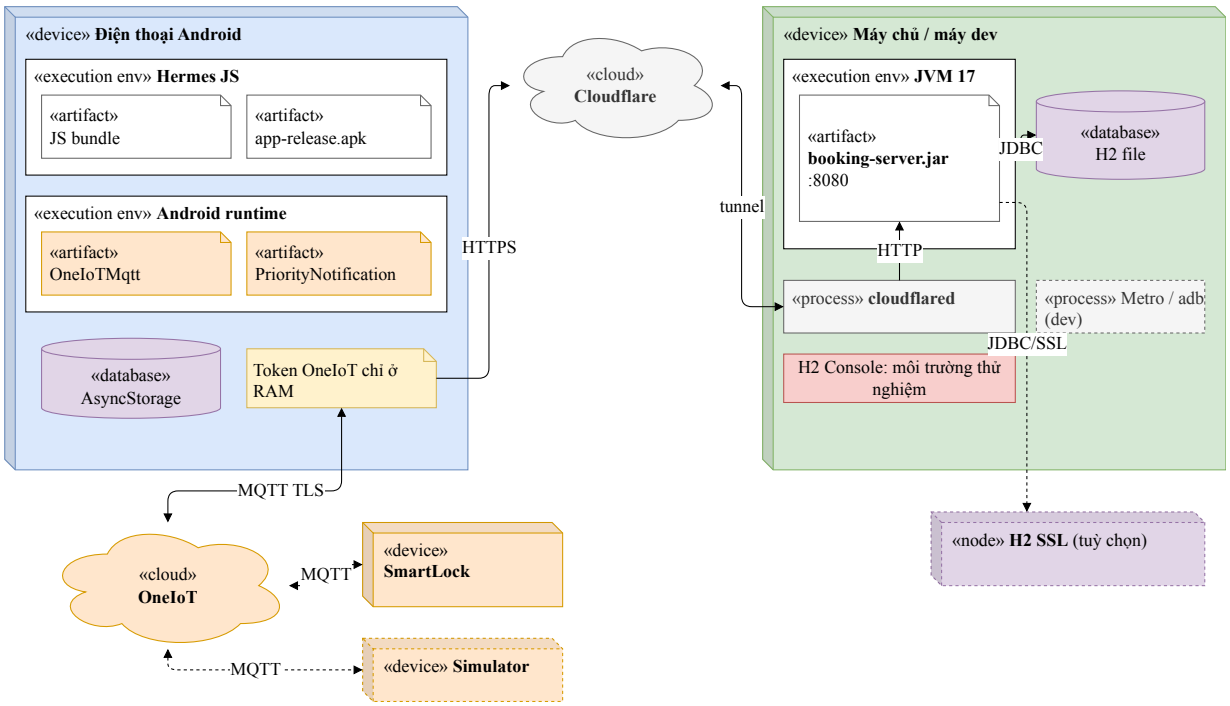
lớp giao diện dùng chung cung cấp các thành phần tái sử dụng. Các module cộng tác qua service và hợp đồng dữ liệu, trong đó nghiệp vụ đặt phòng phối hợp với cấu hình, bảo trì, cấp quyền và thông báo.

Bảng 3.3: Trách nhiệm các lớp Java và dữ liệu trao đổi

Nhóm lớp	**Trách nhiệm đã triển khai**
BookingServerApplication, *DataSeeder*	Khởi động dịch vụ và bổ sung dữ liệu trình diễn, không đại diện cơ sở vật chất thực tế.
SessionAuthFilter, *RequestActor*	Tra token Bearer thành tài khoản, đặt actor trong request và cung cấp actor cho controller.
Sáu controller	Nhận route/body, chuyển actor và dữ liệu tới service, tạo phản hồi HTTP. Các controller dùng chung lớp BookingService.
BookingService	Xác thực/quyền, các nghiệp vụ và giao dịch; dùng EntityManager để truy vấn, khóa và ghi dữ liệu.
AutoApprovalPolicy	Quy tắc tự duyệt dùng chung ở luồng tạo booking phía server.
Domain và converter	11 entity, cấu trúc nhúng, chuyển danh sách/đối tượng lồng thành JSON.
Record đầu vào/kết quả	Hợp đồng request/response cho phòng, booking, PIN, cấu hình liên quan; tách dữ liệu trao đổi khỏi tham số HTTP.
ApiExceptionHandler	Chuyển lỗi dữ liệu thành 400, phiên thành 401, quyền thành 403 và lỗi còn lại thành 500, kèm thông điệp JSON.

3.2.3 Triển khai trên điện thoại và các kết nối

Hình 3.4 mô tả bố trí triển khai: APK/bundle/native chạy trên Android; Java/H2 chạy trên máy chủ; kết nối OneIoT đi từ app tới broker. Metro và ADB phục vụ phát triển hoặc kết nối thử; bản Release local không cần Metro để khởi chạy. Cloudflare và H2 SSL là lựa chọn môi trường, không phải thành phần bắt buộc của bản local.



Hình 3.4: Bố trí triển khai ứng dụng Android, backend và kết nối SmartLock

Ứng dụng Android kết nối trực tiếp OneIoT qua native MQTT để gửi lệnh và tiếp nhận sự kiện từ SmartLock. Backend cung cấp dữ liệu nghiệp vụ qua REST khi bật chế độ remote; chế độ

local lưu dữ liệu trên thiết bị. Simulator và gateway là các chương trình hỗ trợ riêng. Các thành phần app và SmartLock đã được nhóm triển khai, chạy thử và hoàn thành các chức năng thực tế.

3.3 Mô hình dữ liệu và giao tiếp dịch vụ

3.3.1 Thực thể, thuộc tính và quan hệ logic

Mô hình dữ liệu gồm tài khoản, tài nguyên phòng và các bản ghi sử dụng, tiếp cận, bảo trì. Các thực thể liên hệ bằng định danh chuỗi; Bảng 3.4 tổng hợp thuộc tính và quan hệ chính. Lược đồ ER chi tiết nằm ở Phụ lục A.

Bảng 3.4: Mười một thực thể và quan hệ nghiệp vụ chính

Thực thể / bảng	Thuộc tính chính	Liên hệ và điều kiện
Account / accounts	username, password, recoveryCode, role, active.	Username là định danh; trạng thái và vai trò quyết định đăng nhập/quyền.
Profile / user_profiles	username, thông tin họ tên, liên hệ, đơn vị.	Hồ sơ theo tài khoản; user không sửa hồ sơ người khác qua API.
AuthSession / auth_sessions	token, username, createdAt.	Server tra token để nhận diện actor, không tin username do client tự gửi.
Room / rooms	id, name, floor, capacity, equipment, lockType, status, requiresApproval.	Một phòng có nhiều booking/bảo trì theo thời gian; dấu đặc biệt làm phòng luôn cần admin duyệt.
Booking / bookings	requesterUsername, roomId, date/start/end, purpose, status, metadata duyệt và chuỗi lặp.	Tâm điểm liên kết tài khoản, phòng, PIN, nhận chìa, sự kiện, đổi phòng và yêu cầu bảo trì.
PinPermission / pin_permissions	username, roomId, active, thông tin cấp/thu hồi.	Quyền cho một cặp tài khoản–phòng, tách khỏi vai trò admin và vẫn phải kiểm booking.
ManagedSmartLock / managed_smart_locks	assignedRoomId, thông tin khóa/kết nối, nextPasswordId.	Một khóa quản lý gắn tối đa một phòng PIN; password ID quay vòng 1–255.
Maintenance / maintenance_records	roomId, date/start/end, reason, cancelledAt.	Lịch hiệu lực chặn khoảng phòng; hủy lịch không xóa lịch sử.
MaintenanceRequest / maintenance_requests	bookingId, roomId, requesterUsername, reason, status, maintenanceRecordId.	Yêu cầu chờ xử lý khác lịch đã được admin lên.
Notification / notifications	username, title, message, createdAt, read.	Đọc/đánh dấu theo người nhận của phiên.
Configuration / app_configuration	Cửa sổ ngày, hạn mức, các cutoff/grace, tự duyệt và thông báo.	Bản ghi cấu hình chung được đọc ở các bước kiểm nghiệp vụ.

PIN tạm được nhúng trong Booking bằng *@Embedded*; thông tin hẹn/đề xuất nhận chìa, ủy quyền, đổi phòng và danh sách sự kiện dùng cấu trúc lồng hoặc converter JSON. Các quan hệ trên ERD biểu diễn tham chiếu logic theo định danh; tính hợp lệ của các liên hệ được kiểm ở tầng nghiệp vụ, thay cho ánh xạ toàn bộ bằng quan hệ đối tượng JPA.

Dữ liệu thời gian trao đổi gồm ngày dạng YYYY-MM-DD, giờ HH:mm và các mốc ISO/epoch tùy trường. Service Java dùng múi giờ Việt Nam khi kiểm nghiệp vụ, còn local dựng thời điểm theo thiết bị. Đồng hồ và múi giờ ảnh hưởng trực tiếp đến đặt cùng ngày, no-show và khoảng hiệu lực PIN.

3.3.2 Hợp đồng REST và xử lý lỗi

Giao tiếp REST được tổ chức theo sáu controller nghiệp vụ. Các nhóm API bao gồm tài khoản, phòng, booking, lịch lặp, quyền truy cập, nhận chìa, điểm danh, bảo trì và thông báo. Bảng 3.5 tổng hợp 51 cặp phương thức/đường dẫn thành 20 nhóm thao tác.

Bảng 3.5: Tổng hợp endpoint, dữ liệu và quyền truy cập

Phương thức / đường dẫn	**Dữ liệu và kiểm soát**
POST /api/auth/login; *POST /api/auth/register*	Login nhận username/password, trả token/username/role. Đăng ký kiểm dữ liệu và trả phiên; không cần token trước.
POST /api/auth/change-password; *POST /api/auth/reset-password*	Đổi mật khẩu dùng actor/mật khẩu hiện tại; reset dùng username/mã khôi phục. Reset là ngoại lệ không bắt token tại filter.
GET,POST /api/accounts; *PUT,DELETE /api/accounts/{username}*	Admin quản lý tài khoản; service chặn các thao tác vi phạm quyền hoặc điều kiện dữ liệu đang dùng.
GET,PUT /api/profiles/{username}	Hồ sơ gắn tài khoản; service kiểm quyền truy cập theo actor.
GET,POST /api/rooms; *PUT,DELETE /api/rooms/{id}*	Danh sách cho phiên hợp lệ; ghi phòng cần admin, kiểm tên/tầng/sức chứa/loại khóa.
GET,PUT /api/configuration	Đọc quy tắc; admin ghi. Client cũ bỏ qua trường mới không làm mất giá trị hiện có theo code xử lý.
GET /api/room-occupancy	Trả các khoảng sử dụng để tính trạng thái phòng theo lịch, không chỉ theo cờ AVAILABLE.
GET,POST /api/bookings	Đọc theo phạm vi tài khoản; tạo nhận phòng/ngày/giờ/mục đích/lặp, kiểm role và quy tắc trước ghi.
POST /api/bookings/{id}/review; *POST /api/bookings/{id}/change-room*	Admin duyệt/từ chối hoặc đổi phòng; kiểm trạng thái, quyền và điều kiện áp dụng.
POST /api/bookings/{id}/cancel	User chủ lượt; kiểm trạng thái/thời hạn, chuyển CANCELLED và đánh dấu PIN thu hồi trong dữ liệu; không xóa quyền phòng toàn cục.
POST /api/booking-series/{seriesId}/review; *PUT /api/booking-series/{seriesId}/pending*; *POST /api/booking-series/{seriesId}/cancel*	Xử lý các lượt trong chuỗi theo trạng thái; không xem thao tác chuỗi là quyền bỏ qua xung đột từng lượt.
GET,POST /api/pin-permissions; *POST /api/pin-permissions/revoke*	Đọc/cấp/thu hồi quyền theo user–phòng; thao tác quản trị kiểm role trong service.
POST /api/bookings/{id}/temporary-pin	Lưu/cập nhật mã, khoảng hiệu lực và trạng thái gửi; kiểm chủ lượt/admin, booking đã duyệt và quyền liên quan.
GET /api/smart-lock; *POST /api/smart-lock/assign*; *POST /api/smart-lock/unassign*; *POST /api/smart-lock/reserve-password-id*	Quản lý khóa/phòng và cấp ID; cấp ID kiểm phòng khớp, khóa dữ liệu và quyền user nếu actor không phải admin.
POST /api/bookings/{id}/pickup/propose; *POST /api/bookings/{id}/pickup/schedule*; *POST /api/bookings/{id}/pickup/accept*; *PUT /api/bookings/{id}/delegate*	Trao đổi thời gian/địa điểm nhận chìa và người nhận hộ; service kiểm quyền và thứ tự trạng thái thương lượng.
POST /api/bookings/access-event; *POST /api/bookings/{id}/check-in*; *POST /api/bookings/{id}/no-show*	Ghi sự kiện khóa hoặc xác nhận điểm danh theo quyền; kiểm thời gian và bằng chứng có mặt/no-show.
GET,POST /api/maintenance; *POST /api/maintenance/{id}/cancel*	Đọc/lập/hủy lịch bảo trì; lập lịch cần admin và kiểm xung đột.
GET,POST /api/maintenance/requests; *POST /api/maintenance/requests/{id}/schedule*; *POST /api/maintenance/requests/{id}/reject*	Yêu cầu từ booking đang dùng, được admin lên lịch hoặc từ chối.

Tổng hợp endpoint, dữ liệu và quyền truy cập (tiếp)

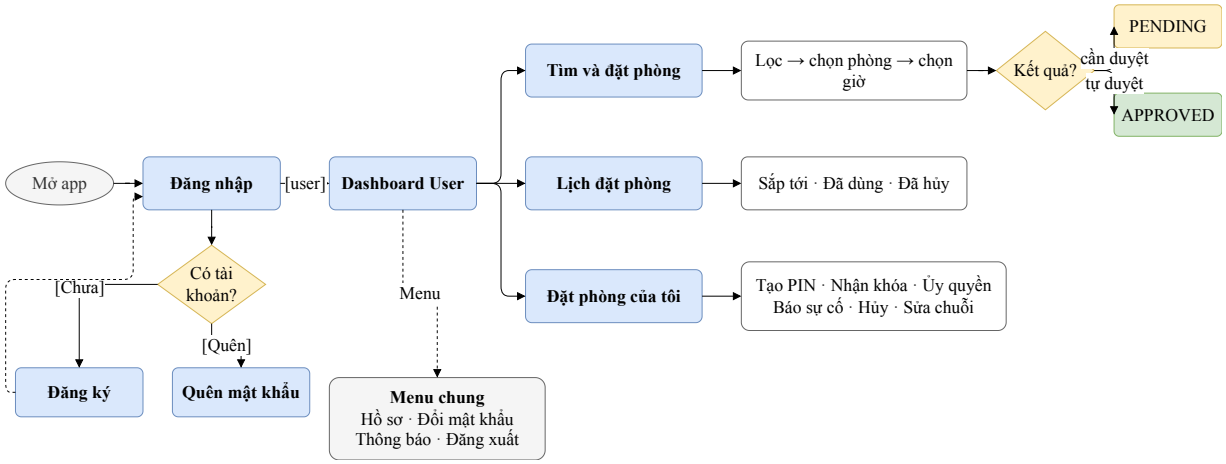
Phương thức / đường dẫn	**Dữ liệu và kiểm soát**
POST /api/bookings/{id}/room-observation	Gửi ghi chú cho admin, không tự tạo yêu cầu hoặc lịch bảo trì.
GET,POST /api/notifications; *POST /api/notifications/read*	Đọc/đánh dấu theo phiên; tạo thông báo còn phải qua kiểm tra quyền service.

SessionAuthFilter bỏ qua ba route login/register/reset-password; các route API còn lại cần token Bearer hợp lệ. User không được chọn username người khác để mở rộng dữ liệu trả về. Client bọc fetch, thêm Authorization, xử lý JSON/204/401 và timeout 12 giây; API lỗi dùng trường *{message}*. Kết quả kiểm thử API và tích hợp được trình bày ở Chương 5.

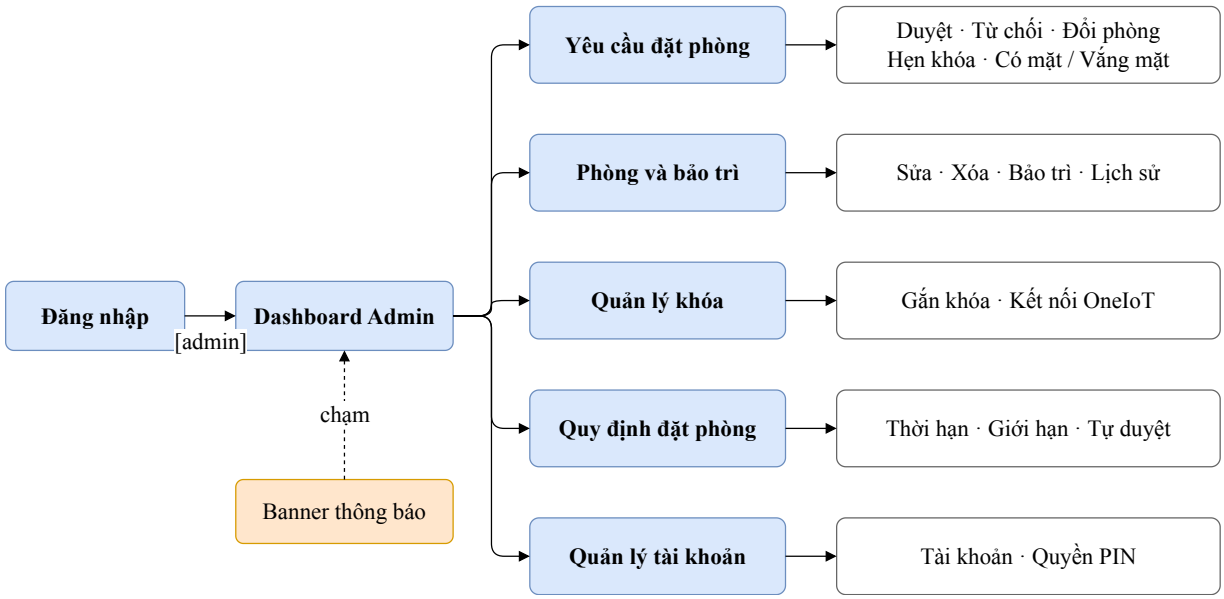
3.4 Thiết kế giao diện và điều hướng

3.4.1 Luồng màn hình theo vai trò

Giao diện Android dọc dùng dashboard, đầu trang, Back, chuông và menu tài khoản. Tìm phòng theo tầng/ngày/giờ và gửi yêu cầu trên cùng màn hình. Hình 3.5–3.6 mô tả điều hướng hai vai trò; ảnh thực nghiệm ở Chương 4.



Hình 3.5: Luồng màn hình và các tác vụ chính của người dùng



Hình 3.6: Luồng màn hình quản trị và các điểm vào từ thông báo

Bảng 3.6: Màn hình, mục đích và dữ liệu điều hướng

Điểm vào	**Trách nhiệm / tham số**
Đăng nhập, đăng ký, khôi phục mật khẩu	Chọn form xác thực qua selectedFeature trong LoginScreen; khi đăng nhập thành công, callback chuyển session lên App.
Dashboard User	Mở Tìm và đặt phòng, Lịch đặt phòng, Đặt phòng của tôi; truyền username và onBack cho màn con.
Dashboard Admin	Mở Yêu cầu đặt phòng, Quản lý tài khoản, Quản lý phòng và bảo trì, Quản lý khóa, Quy định đặt phòng; truyền thông tin admin khi cần.
Menu tài khoản và chuông	Mở hồ sơ, đổi mật khẩu, thông báo; logout xóa phiên trong UI. Banner/notification có thể đưa admin tới màn xử lý yêu cầu.
Đặt phòng của tôi	Phân nhóm lượt theo thời gian/trạng thái; mở thao tác sửa/hủy chuỗi, PIN, nhận chìa, ủy quyền và phản ánh phòng.
Quản lý phòng và bảo trì	Chọn tầng/phòng; chỉnh thông tin, xem lịch hoặc lập/xử lý bảo trì trên cùng màn hình.

3.4.2 Cơ chế chuyển màn, làm mới và quay lại

Bảng điều khiển dùng trạng thái *activeScreen* để chọn màn hình hiển thị. Props mang dữ liệu và callback thay route params. Khi đóng màn con, *closeChildScreen* đặt màn đang chọn về null rồi làm mới số thông báo/yêu cầu. BackHandler đóng menu trước, sau đó quay về dashboard; listener được tháo khi effect thay đổi hoặc unmount.

App yêu cầu đăng nhập lại khi khởi động; các màn hình được điều khiển bằng state và callback. Loading/error và cleanup listener/timer là trách nhiệm từng luồng bất đồng bộ.

3.5 Dữ liệu cá nhân, quyền thiết bị và các ràng buộc

AndroidManifest khai báo INTERNET, POST_NOTIFICATIONS và VIBRATE. Trên Android từ API 33, app kiểm và xin quyền thông báo khi chạy [13]; từ chối quyền không đồng nghĩa nghiệp vụ đặt phòng bị cấm, nhưng thông báo hệ điều hành có thể không hiện. Ứng dụng không xin quyền vị trí, camera hoặc micro cho luồng đặt phòng hiện tại.

Token REST và OneIoT giữ trong RAM của phiên. Token OneIoT không được ghi vào source

hoặc Async Storage; app ngắt MQTT khi đăng xuất hoặc xuống nền, việc nhận sự kiện khóa diễn ra trong phiên MQTT đang hoạt động. Mật khẩu/PIN và một phần dữ liệu khôi phục hiện còn dạng rõ trong local/entity server. Mã hóa gói lệnh khóa không giải quyết việc bảo vệ các dữ liệu lưu này.

Native TLS đang bỏ qua xác minh chứng thư; Android cho phép cleartext để thử HTTP. Hướng gia cố bảo mật gồm xác thực, bảo vệ lưu trữ, chứng thư và phân quyền khi mở rộng triển khai. H2 console và đường tunnel trong sơ đồ là cấu hình thử nghiệm cần quản lý theo môi trường. Chế độ local dùng được khi thiếu backend, nhưng đó là chế độ chọn trước khi build, không phải cơ chế tự fallback/offline sync của REST.

3.6 Các giải pháp kỹ thuật cho luồng nghiệp vụ cốt lõi

Ba nhóm giải pháp tập trung vào tính nhất quán lịch, quyền tiếp cận và tổ chức dữ liệu. Chúng giải thích cách các bước kiểm tra, trạng thái và ranh giới dữ liệu hỗ trợ luồng nghiệp vụ cốt lõi.

3.6.1 Chống trùng lịch, giao dịch và quyết định duyệt

Bài toán. Hai lượt đặt có thể không trùng khi đọc riêng nhưng xung đột khi cùng được ghi. Cần phân biệt khoảng chạm biên với khoảng giao nhau; đồng thời kiểm lại phòng khi admin duyệt vì lịch bảo trì hoặc lượt đã duyệt có thể thay đổi trong thời gian chờ.

Phép kiểm khoảng. Với cùng ngày, hai khoảng nửa mở *[s1,e1)* và *[s2,e2)* giao nhau khi đồng thời có $s1 < e2$ và $s2 < e1$. Dùng bất đẳng thức chặt nên 08:00–09:00 và 09:00–10:00 được xem là liền kề. Ngày khác nhau không giao trong phép kiểm này. Dữ liệu thời gian được chuẩn hóa và kiểm tra trước phép so sánh; khoảng có giờ bắt đầu lớn hơn hoặc bằng giờ kết thúc bị từ chối.

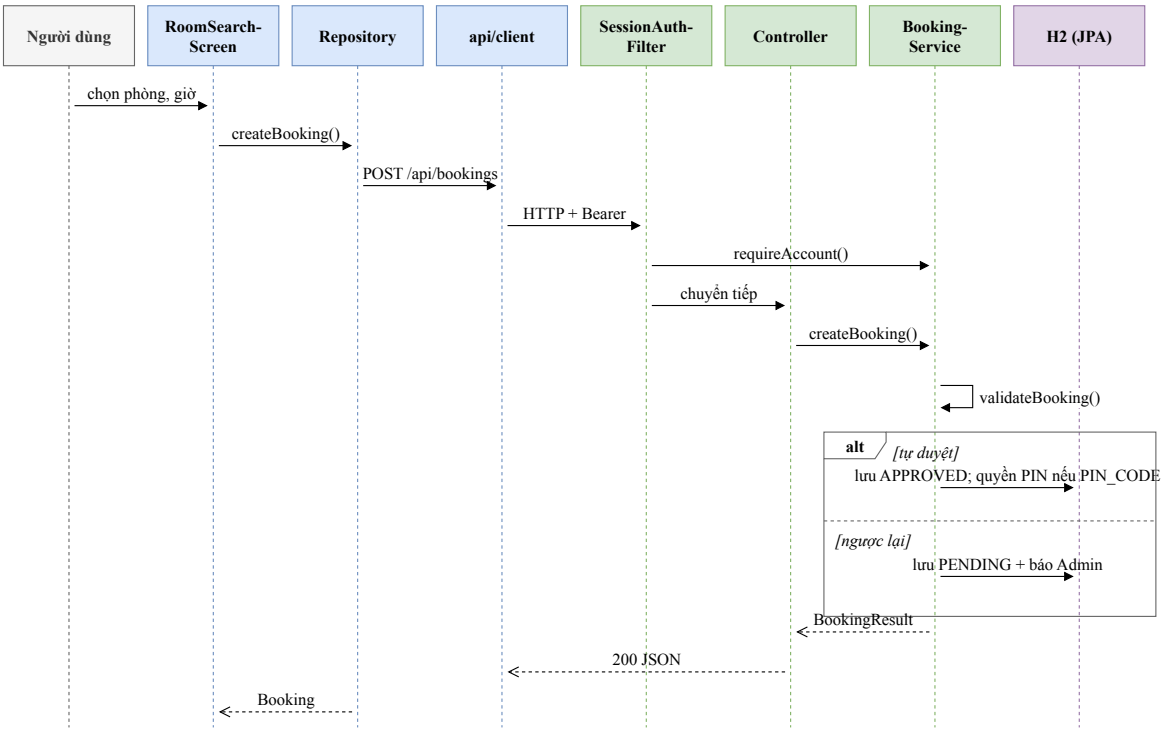
Nghiệp vụ đặt phòng và bảo trì dùng chung phép kiểm khoảng; backend thực hiện điều kiện tương ứng trong tầng dịch vụ. Khi tạo, hệ thống xét phòng, lịch của chính user và bảo trì; khi duyệt một lượt đang chờ, code kiểm lại thời gian kết thúc, bảo trì và xung đột với lượt đã duyệt của phòng. Kiểm tra khi tạo bảo đảm yêu cầu hợp lệ; kiểm tra khi duyệt bảo đảm quyết định dựa trên lịch và bảo trì hiện thời.

Thứ tự giao dịch ở backend. *BookingService.createBooking* dùng *@Transactional*, khóa Configuration và Room bằng *PESSIMISTIC_WRITE* trước khi đọc lịch. Với chuỗi lặp, từng lượt được sinh, kiểm và bổ sung vào tập xét để phát hiện cả xung đột trong chuỗi (Bảng 3.7).

Bảng 3.7: Thứ tự tạo booking trong giao dịch Java

Bước	Xử lý
1	Kiểm actor là user và không mạo danh requesterUsername khác.
2	Khóa cấu hình chung và bản ghi phòng được chọn trước khi đọc lịch.
3	Kiểm định dạng ngày và số tuần lặp; sinh ngày của từng lượt nếu đặt theo chuỗi.
4	Validate từng lượt: phòng, giờ, bảo trì, chồng phòng/user, mục đích, cửa sổ và hạn mức theo điều kiện của lượt đầu/lượt lặp.
5	Tạo booking với trạng thái ban đầu PENDING; lưu metadata chuỗi và bổ sung vào tập đang xét.
6	Sau validation, AutoApprovalPolicy quyết định có tự duyệt: chỉ khi công tắc bật và phòng không yêu cầu duyệt riêng.
7	Nhánh tự duyệt cập nhật APPROVED, người duyệt system, quyền PIN nếu phòng PIN và thông báo; nhánh thủ công giữ PENDING và báo admin.
8	Trả BookingResult trong giao dịch. Lỗi dữ liệu/điều kiện được ném thành ngoại lệ; không trả kết quả thành công cho lượt vi phạm.

reviewBooking khóa Configuration và booking đích; khi duyệt sẽ kiểm trạng thái PENDING và các điều kiện hiện thời rồi cập nhật quyết định. Quyền PIN được cấp cho phòng khóa số sau khi duyệt. Sơ đồ trình tự Hình 3.7 bổ sung điểm gọi qua UI, client HTTP, filter, controller và service; thứ tự khóa và kiểm tra dữ liệu được tổng hợp trong Bảng 3.7.



Hình 3.7: Trình tự tạo booking qua REST và nhánh tự duyệt/thủ công

Kết quả và hạn chế kỹ thuật. Các ca kiểm thử ở Chương 5 xác nhận xử lý đúng trùng lịch, chạm biên, bảo trì và hạn mức. Backend khóa bản ghi cấu hình/phòng trước khi kiểm và ghi dữ liệu. Khóa cấu hình chung khiến các thao tác dùng cùng tài nguyên phải chờ nhau, là điểm cần tối ưu khi mở rộng. Kho local đọc/ghi JSON, còn backend quản lý giao dịch và khóa dữ liệu bằng JPA.

3.6.2 Cấp quyền theo phòng và vòng đời PIN tạm thời

Bài toán. Vai trò user không được biến thành quyền tạo mã ở mọi phòng. Một người chỉ được tạo PIN cho booking đã duyệt của mình, tại phòng được phép và đang gắn khóa phù hợp. Kết nối OneIoT có thể chưa có khi tạo mã, vì vậy lưu mã và gửi lệnh cần là các trạng thái riêng.

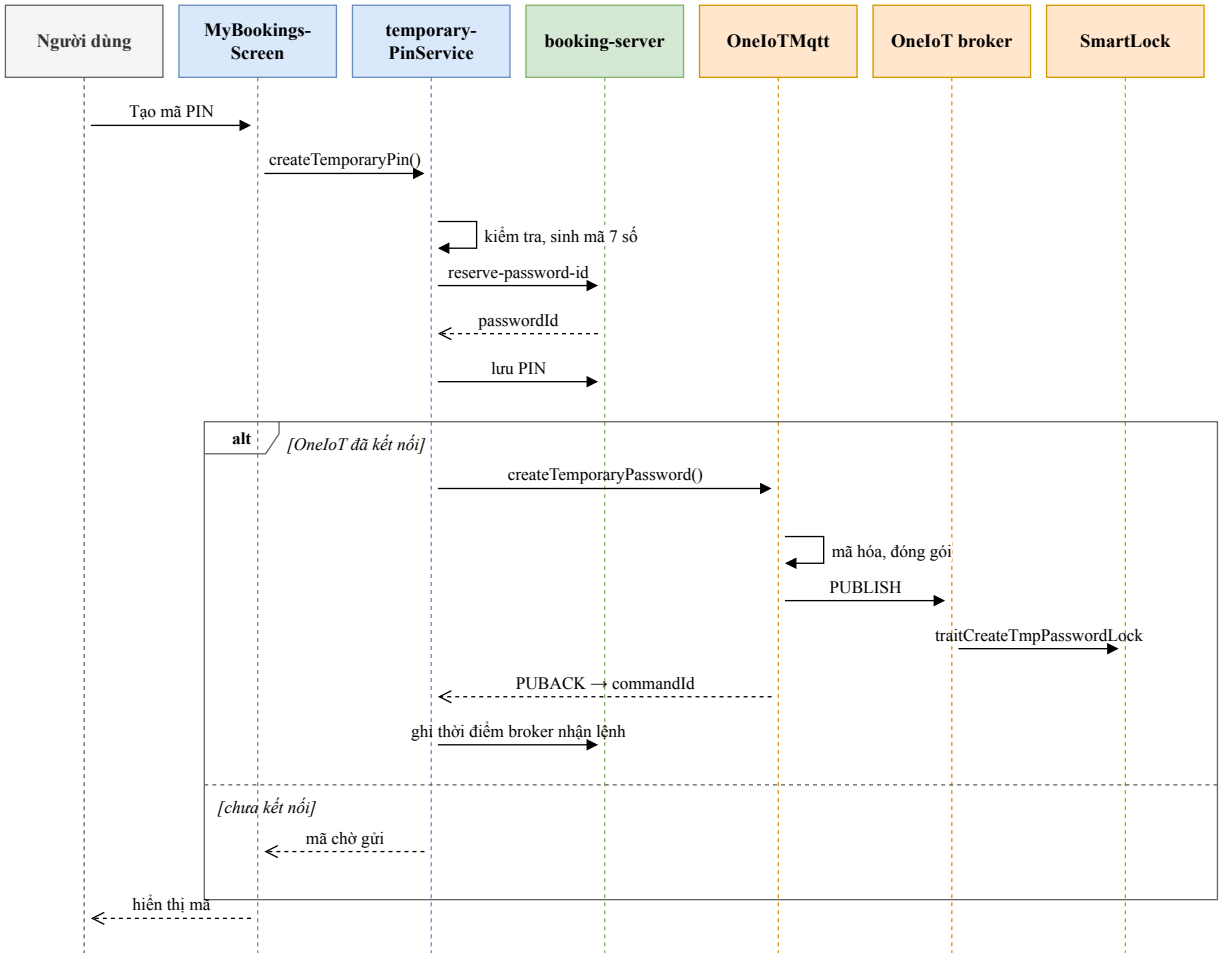
Kiểm quyền ở các tầng. Dịch vụ cấp PIN kiểm vai trò, booking tồn tại/APPROVED, phòng PIN, chủ lượt/quyền theo phòng đối với user, chưa có PIN chưa thu hồi và lượt chưa kết thúc. Admin có quyền thao tác phù hợp nhưng vẫn phải đáp ứng các điều kiện nghiệp vụ của booking/phòng. Khi duyệt tay hoặc tự duyệt phòng PIN, hệ thống cấp quyền cho cặp username–roomId. Thu hồi quyền theo phòng tách khỏi thay đổi role tài khoản.

Mã mới chỉ được tạo khi lượt đặt không còn PIN chưa thu hồi. Do đó, PIN hết thời gian nhưng chưa ghi nhận thu hồi vẫn ngăn việc cấp mã tiếp theo.

Khoảng hiệu lực và ID khóa. Mã nghiệp vụ có đúng bảy chữ số. Với khoảng sử dụng, validFrom được tính bằng giờ bắt đầu trừ khoảng đệm; validUntil bằng giờ kết thúc cộng khoảng

đệm. Giá trị mặc định là 10 phút và đọc từ cấu hình, không gắn cứng trong UI. Trước khi lưu, service xin password ID của khóa gắn phòng; ID nằm trong 1–255 và quay vòng sau 255. Backend khóa bản ghi khóa khi cấp ID. Cấp ID, lưu PIN và gửi MQTT là các lời gọi riêng trong quy trình.

Lưu trước, gửi khi có phiên. App lưu code, thời hạn và password ID vào booking trước. Nếu OneIoT chưa nối, người dùng vẫn xem được mã và trạng thái chờ. Khi có phiên, *deliverPendingTemporaryPins* duyệt các PIN chưa thu hồi/chưa gửi, booking còn APPROVED, đúng phòng đang gắn khóa và chưa hết hạn. Một lần gửi thất bại được ghi thông tin lần thử/lỗi để UI phân biệt với chưa thử gửi.



Hình 3.8: Tạo, lưu và gửi PIN; xác nhận publish của broker tách với thực thi trên khóa

Native *OneIoTMqttPackage.kt* chuyển thời hạn sang định dạng bản tin, mã hóa phần mật khẩu và gửi trait *traitCreateTmpPasswordLock*. Sau publish được broker xác nhận, app lưu *lockDeliveredAt* và request ID. Giao diện cập nhật trạng thái gửi theo kết quả publish; các bản tin sự kiện khóa được xử lý qua luồng nhận MQTT.

Kết quả và hạn chế kỹ thuật. Các kiểm thử access control và native bridge xác nhận đúng dữ liệu, quyền, thời hạn và cơ chế gửi lại. Ứng dụng và SmartLock thực hiện luồng cấp, sử dụng PIN thực tế. Trạng thái revokedAt quản lý PIN trong booking. Hướng gia cố gồm quản lý ID hữu hạn, bảo vệ dữ liệu, nguồn sinh mã và phản hồi thiết bị.

3.6.3 Cùng giao diện nghiệp vụ, hai chế độ dữ liệu

Bài toán. Hệ thống hỗ trợ vận hành độc lập trên một thiết bị và sử dụng dữ liệu dùng chung qua REST. Hai chế độ dùng cùng giao diện nghiệp vụ để giảm lặp xử lý; nguồn dữ liệu được chọn rõ ràng trong cấu hình bản build.

Giải pháp triển khai. Repository giữ các hàm nghiệp vụ mà UI gọi; *isRemoteApiEnabled* chọn nhánh REST hoặc đọc/ghi JSON local. Client tập trung thêm token và chuyển lỗi mạng/HTTP thành Error cho màn hình. Kiểu TypeScript và cấu trúc request/response Java xác định hợp đồng trao đổi; kiểm thử API kiểm tra phương thức, dữ liệu, phiên và các nhánh lỗi.

Bảng 3.8: Đối chiếu thao tác local và REST

Thao tác	Local	REST
Xác thực	Đọc tài khoản của kho thiết bị, tạo session UI.	Login tạo token/actor tại server; request sau mang Bearer.
Tạo/duyệt booking	Repository kiểm quy tắc trên các tập dữ liệu local và ghi JSON.	API/service kiểm actor và dữ liệu trong giao dịch; server là nơi quyết định trạng thái được lưu.
Phòng/cấu hình/bảo trì	Mỗi máy giữ danh mục và cấu hình riêng.	Các client gọi cùng server sử dụng chung danh mục, cấu hình và dữ liệu phòng.
Quyền và PIN	Quyền và PIN gắn tài khoản/phòng/booking trong kho local.	Quyền và PIN lưu ở server; việc gửi MQTT vẫn xuất phát từ app/native, không chuyển cho Java.
Thông báo	Lưu theo người nhận trong thiết bị; banner chỉ hiện cho tài khoản đang đăng nhập.	Server lưu theo user; client tải thông báo qua REST bằng phiên đăng nhập.
Lỗi kết nối	Không cần booking server; OneIoT vẫn là kết nối riêng nếu dùng khóa.	Trả lỗi/timeout theo client; không tự fallback hoặc tải backlog local.

Kết quả và hạn chế kỹ thuật. Một nhóm màn hình phục vụ cả hai chế độ; thay đổi cấu hình chọn nguồn dữ liệu mà không chuyển dữ liệu giữa hai kho. Nếu bổ sung offline sync sau này, cần thiết kế định danh, hàng đợi, phiên bản dữ liệu, quy tắc hòa giải và kiểm thử độc lập. Kiểm thử hợp đồng REST duy trì cấu trúc trao đổi giữa app và server; khi mở rộng cần quản lý kết nối, tính sẵn sàng của dịch vụ và dữ liệu dùng chung.

CHƯƠNG 4. TRIỂN KHAI ỨNG DỤNG

4.1 Môi trường và cấu trúc mã nguồn

Ứng dụng dùng Community CLI, không dùng Expo. Các thành phần gồm React Native 0.86.2, React 19.2.3, Async Storage 3.1.1, TypeScript 5.9.3 và Jest 29. package.json yêu cầu Node từ 22.11.0. Android cấu hình SDK 36, Kotlin 2.1.20 và native module riêng; backend dùng Java 17/Maven/Spring Boot 3.4.4.

4.1.1 Cấu hình công cụ và điều kiện tái lập

Bảng 4.1 trình bày công cụ và cấu hình để cài đặt, build và tái lập ứng dụng. Hệ điều hành của thiết bị chạy thử được nêu tại mục 5.1.

Bảng 4.1: Công cụ và cấu hình biên dịch

Hạng mục	Cấu hình biên dịch	Tệp đối chiếu
Node/npm	Node từ 22.11.0; npm cài theo lockfile	*code/booking_classroom/package.json*; *package-lock.json*.
Android	Compile/target SDK 36; min SDK 24; Build-Tools 36.0.0	*android/build.gradle*; *android/app/build.gradle*, tính từ thư mục app.
Native build	NDK 27.1.12297006; Kotlin 2.1.20; Gradle wrapper 9.3.1	*android/build.gradle*; *android/gradle/wrapper/gradle-wrapper.properties*.
Bản Android	Package com.bookingclassroom; versionName 1.0; versionCode 1	*android/app/build.gradle*; version npm 0.0.1 là trường riêng trong package.json.
Backend	Java 17; Spring Boot 3.4.4; H2 file mặc định	*code/booking-server/pom.xml*; *src/main/resources/application.properties*.

Máy build cần JDK, Android SDK và Platform-Tools/ADB được cấu hình đúng đường dẫn; Windows dùng PowerShell cho script hiện có. Các phiên bản và tên thiết bị trong hồ sơ kiểm thử được trình bày riêng ở Chương 5.

4.1.2 Cấu trúc mã nguồn và trách nhiệm triển khai

Các đường dẫn trong chương tính từ repository *final_project/*. Mã nguồn ứng dụng ở *code/booking_classroom/*, backend ở *code/booking-server/*. Cấu trúc thực tế được tóm tắt như sau:

Bảng 4.2: Cấu trúc thư mục và trách nhiệm

Vị trí trong repository	Vai trò
code/booking_classroom/App.tsx	Điểm ghép giao diện và phiên của app.
code/booking_classroom/src/app/	Ghép màn hình theo vai trò.
code/booking_classroom/src/core/	API, storage, time, config và types dùng chung.
code/booking_classroom/src/modules/	11 module nghiệp vụ.
code/booking_classroom/src/shared/	Thành phần UI dùng chung.
code/booking_classroom/android/app/src/	Native Android.
code/booking_classroom/__tests__/; *code/booking_classroom/quality-tests/*	Test ứng dụng và bộ kiểm thử chất lượng riêng.
code/booking-server/src/main/java/	Gói com/bookingclassroom/server/: service, entity, session; thư mục api/ có sáu controller.

Cấu trúc thư mục và trách nhiệm (tiếp)

Vị trí trong repository	Vai trò
code/booking-server/src/main/resources/	Cấu hình backend.
code/booking-server/src/test/	Kiểm thử tích hợp H2.

Màn hình tiếp nhận thao tác và hiển thị dữ liệu; dịch vụ kiểm tra nghiệp vụ trước khi đọc/ghi qua repository. Model xác định kiểu dữ liệu trao đổi. Nghiệp vụ đặt phòng phối hợp với phòng, cấu hình, bảo trì, cấp quyền, tự duyệt và thông báo; phép kiểm giao thời gian được dùng chung để giữ nhất quán.

4.2 Ánh xạ yêu cầu chức năng tới mã nguồn

Trong Bảng 4.3, tiền tố file app là *code/booking_classroom/*; phần tên hàm mô tả điểm triển khai cụ thể.

Bảng 4.3: Ánh xạ chức năng, vị trí và bằng chứng

FR	Màn hình / vị trí xử lý	Hành vi và minh chứng
FR01	*src/modules/auth/screens/LoginScreen.tsx*; *src/modules/auth/services/accountRepository.ts*.	Đăng nhập cập nhật phiên và mở màn hình đúng vai trò; đăng ký, đổi/khôi phục mật khẩu kiểm dữ liệu và phản hồi lỗi. TC01, TC17; D01–D03.
FR02	*src/modules/room_management/screens/RoomSearchScreen.tsx*; *src/modules/room_management/services/roomRepository.ts*.	Bộ lọc kết hợp danh mục, booking và bảo trì để hiển thị phòng khả dụng. Hình 4.2; TC02, TC12.
FR03	*src/modules/booking/services/bookingRepository.ts*; *src/modules/booking/screens/MyBookingsScreen.tsx*.	Kiểm từng lượt trước khi lưu chuỗi; hủy/sửa theo quyền, trạng thái và hạn chót. TC11–TC12, TC21.
FR04	*src/modules/booking/screens/AdminBookingScreen.tsx*; *src/modules/auto_approval/model/approvalPolicy.ts*.	Duyệt tay theo quyền admin; tự duyệt sau kiểm tra hợp lệ khi phòng thường và cấu hình cho phép. TC07, TC20, TC24.
FR05	*src/modules/access_control/services/temporaryPinService.ts*; *src/modules/booking/services/bookingRepository.ts*.	Kiểm chủ lượt/quyền phòng, lưu PIN và gửi qua OneIoT; trao đổi lịch nhận chìa và ủy quyền. TC03, TC10, TC13–TC15, TC19.
FR06	*src/modules/smart_lock/services/smartLockAccessEventService.ts*; *src/modules/booking/services/bookingRepository.ts*.	Tiếp nhận sự kiện khóa, ghi nhận có mặt/ra và chặn no-show khi đã có check-in. TC10, TC22.
FR07	*src/modules/schedule_maintenance/services/maintenanceRepository.ts*; *src/modules/booking/services/bookingRepository.ts*.	Tách ghi chú, yêu cầu bảo trì và lịch chặn phòng; admin xử lý theo trạng thái. TC02, TC23.
FR08	*src/modules/account_management/*; *src/modules/configuration/*; *src/modules/profile/*; *src/modules/notifications/*.	Quản trị theo quyền; lưu hồ sơ/cấu hình và thông báo theo người nhận. TC06, TC08–TC09, TC16, TC18.

Màn tìm và đặt nằm trong room_management nhưng gọi nghiệp vụ booking; quản lý tài khoản có UI riêng nhưng dùng accountRepository của auth. Phân quyền online được kiểm thêm ở filter/service Java, không dựa riêng vào việc UI ẩn nút.

4.3 Ma trận kỹ thuật React Native

Bảng 4.4: Kỹ thuật, cách áp dụng, lý do và kiểm chứng

Kỹ thuật	Vị trí và cách áp dụng / lý do	Bằng chứng
Component, JSX, Props	*App.tsx*: App truyền session/onLogout cho RoleDashboardScreen và onLogin cho LoginScreen. Props/callback tập trung phiên ở gốc, tránh sao chép trạng thái xác thực.	Đoạn A; TC01, D02.
State	*src/modules/auth/screens/LoginScreen.tsx*: LoginScreen dùng useState cho input, error và submitting. Trạng thái cục bộ đáp ứng biểu mẫu; useReducer không áp dụng.	TC01, TC17; D01–D03.
Effect và cleanup	*App.tsx*: App.useEffect đăng ký AppState/sự kiện khóa rồi tháo listener và ngắt MQTT; *src/modules/auth/screens/LoginScreen.tsx*: effect xử lý BackHandler. Cleanup tránh lặp bộ lắng nghe khi chuyển màn.	App, dòng 24–38; kiểm thử bridge và callback.
Custom Hook / Context / Store	Không áp dụng ở tầng nghiệp vụ: quy mô hiện tại dùng state cục bộ, props và service; chưa cần store dùng chung.	Cấu trúc phụ thuộc ở mục 4.1; đoạn A.
Điều hướng	*src/app/screens/RoleDashboardScreen.tsx*: RoleDashboardScreen dùng activeScreen, closeChildScreen và onBack; giữ điều hướng tập trung cho số màn hiện tại.	Hình 3.5–3.6; D02.
Async, API, Promise	*src/core/api/client.ts*: apiRequest dùng fetch, AbortController và finally; tập trung token, giới hạn chờ và chuyển lỗi cho UI.	Đoạn B; TC04–TC05.
Form và validation	*src/modules/auth/screens/LoginScreen.tsx*: submit kiểm dữ liệu, đặt submitting, gọi authenticate và khôi phục trạng thái trong finally; kiểm lại ở service để bảo vệ nghiệp vụ.	TC01, TC17; D01, D03.
Danh sách	*src/modules/room_management/screens/RoomSearchScreen.tsx*: RoomSearchScreen dùng ScrollView/map và key theo định danh, phù hợp danh mục hiện tại. FlatList/phân trang không áp dụng.	Hình 4.2; kiểm thử bộ lọc.
Lưu trữ và quyền	*src/core/storage/jsonStorage.ts*: readJson/writeJson bọc Async Storage; *src/modules/notifications/services/priorityBanner.ts*: requestPriorityNotificationPermission xử lý API 33+. Tách lưu trữ khỏi UI và xin quyền theo nền tảng.	TC08–TC09; D05–D06.
Tối ưu / nền tảng	*src/app/screens/RoleDashboardScreen.tsx*: refreshUnreadCount dùng useCallback để ổn định dependency của effect; *src/modules/notifications/services/priorityBanner.ts* kiểm Platform.OS trước khi gọi native.	Kiểm thử quyền API 31/33 và bản chạy Android.

4.4 Phân tích hai đoạn mã cốt lõi

4.4.1 Đoạn A: giao diện, phiên và callback

Tệp: *code/booking_classroom/App.tsx*, dòng 40–62. Liên quan FR01/FR08 và G03. Đầu vào là session do useState ở dòng 18 giữ; đầu ra là cây component theo trạng thái xác thực.

```
40  return (
41    <SafeAreaProvider>
42      <StatusBar backgroundColor="#F4F7FB" barStyle="dark-content" />
43      <SafeAreaView
44        style={styles.safeArea}
45        edges={['top', 'right', 'bottom', 'left']}
46      >
47        {session ? (
48          <RoleDashboardScreen
49            session={session}
50            onLogout={() => {
51              disconnectOneIoT().catch(() => {});
52              clearApiSession();
53              setSession(null);
54            }}
55          />
56        ) : (
57          <LoginScreen onLogin={setSession} />
58        )}
59      </SafeAreaView>
60      <PriorityBannerHost />
61    </SafeAreaProvider>
62  );
```

Phân tích. Dòng 47 rẽ nhánh bằng session; 48–55 truyền dữ liệu và callback cho màn theo vai trò. Dòng 50–53 gom logout: ngắt OneIoT, xóa token, đặt session null. Dòng 57 truyền setSession làm onLogin; khi LoginScreen hoàn tất authenticate, callback làm App render lại. Props/callback duy trì một chủ sở hữu phiên, không sao chép mật khẩu vào state toàn app.

Trạng thái lỗi/loading thuộc LoginScreen: submit kiểm ô trống, đặt submitting, xử lý null/error rồi khôi phục trạng thái trong finally. Effect tại App dòng 24–38 đăng ký listener và dọn dẹp khi unmount/xuống nền [5]. Người dùng đăng nhập lại sau khi mở ứng dụng; MQTT hoạt động theo phiên kết nối. App.test/authScreens kiểm giao diện và điều hướng; thử nghiệm trên thiết bị xác nhận quá trình sử dụng cùng SmartLock.

4.4.2 Đoạn B: xử lý HTTP bất đồng bộ và ngoại lệ

Tệp: *code/booking_classroom/src/core/api/client.ts*, dòng 35–61. Liên quan FR01–FR08/NFR02. Đầu vào là path, method, body và token của phiên; đầu ra là response JSON hoặc Error cho màn hình.

```
35  let response: Response;
36  const controller = new AbortController();
37  const timeout = setTimeout(() => controller.abort(), REQUEST_TIMEOUT_MS);
38  try {
39    const baseUrl = API_BASE_URL.trim().replace(/\/+$/, '');
40    if (!/^https?:\/\/i.test(baseUrl)) {
41      throw new Error('Địa chỉ máy chủ đặt phòng không hợp lệ.');
42    }
43    response = await fetch(`${baseUrl}${path.startsWith('/') ? path : `/${path}`}`, {
44      method: options?.method ?? 'GET',
45      headers: {
46        Accept: 'application/json',
47        'Content-Type': 'application/json',
48        ...(token ? { Authorization: `Bearer ${token}` } : {}),
49      },
50      body: options?.body === undefined ? undefined : JSON.stringify(options.body),
51      signal: controller.signal,
52    });
53  } catch (error) {
54    if (error instanceof Error && error.message === 'Địa chỉ máy chủ đặt phòng không hợp lệ.') throw
      error;
55    const timedOut = error instanceof Error && error.name === 'AbortError';
56    throw new Error(timedOut
57      ? 'Máy chủ đặt phòng không phản hồi trong 12 giây.'
58      : 'Không kết nối được máy chủ đặt phòng. Hãy kiểm tra địa chỉ và trạng thái server.');
59  } finally {
60    clearTimeout(timeout);
61  }
```

Phân tích. Dòng 36–37 tạo controller và timer giới hạn chờ. Dòng 39–42 chuẩn hóa/kiểm URL; 43–52 đóng gói request, thêm Bearer nếu có, serialize body và gắn signal. Dòng 53–58 chuyển lỗi mạng/abort thành thông điệp nghiệp vụ; finally 59–61 xóa timer dù thành công hay lỗi.

Phần sau trích đoạn xử lý 204, parse JSON, thông báo lỗi HTTP và xóa token khi 401. Client không tự retry để tránh lặp thao tác ghi, không tự fallback local vì hai kho không hòa giải. Kiểm thử HTTP mô phỏng fetch và timer để xác nhận header, body, lỗi JSON, lỗi 401, mất mạng và ngắt yêu cầu sau 12 giây (TC04–TC05). Đây là kiểm tra cơ chế giới hạn chờ ở client.

4.5 Truy vết luồng đăng nhập và xử lý ngoại lệ

Bảng 4.5: Từ thao tác tới render trong luồng đăng nhập

Bước / vị trí	Dữ liệu và trạng thái	Kiểm chứng
1. LoginScreen onChange/submit	TextInput cập nhật username/password. Thiếu ô bắt buộc đặt error; không gửi request.	AuthScreens quality và giao diện đăng nhập.
2. submit / authenticate	submitting=true, xóa/ghi error theo kết quả; local đọc auth.accounts hoặc REST gọi login.	Auth unit/component; apiClient mock.
3. accountRepository / apiRequest	REST nhận token/role; lỗi xác thực trả null theo điều kiện. Mạng lỗi/timeout thành Error, 401 xóa token.	Test sai mật khẩu, duplicate và các ca API lỗi.
4. onLogin / App.setSession	Thành công truyền username/role, render RoleDashboard theo role; không truyền mật khẩu.	App.test; biên bản Admin/User.
5. finally / cleanup	submitting=false để mở lại nút. BackHandler/listener bị tháo khi màn tương ứng đóng; logout/ngắt nền đóng MQTT.	Source submit/useEffect; native bridge được mock.

Khi xác thực thất bại hoặc mất kết nối REST, ứng dụng giữ người dùng ở biểu mẫu và hiển thị thông điệp lỗi cụ thể. Dữ liệu local thiếu được repository khởi tạo theo model; dữ liệu remote lỗi định dạng bị API client từ chối. Các màn giữ loading/error bằng state; cập nhật state làm component render lại.

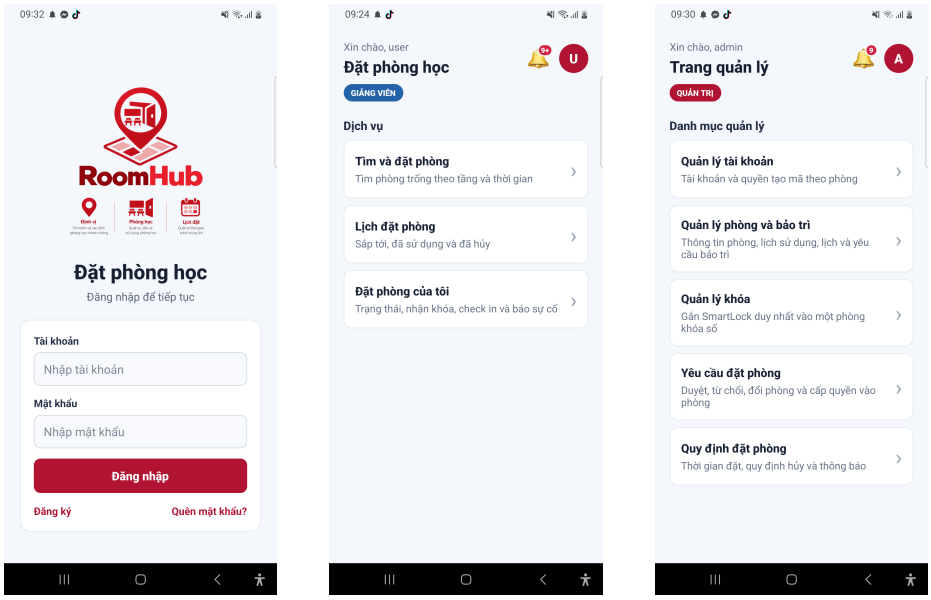
4.6 Giao diện và sản phẩm chạy thử

Chín ảnh giao diện minh họa các bước thao tác và nhóm chức năng của ứng dụng, từ đăng nhập, đặt phòng đến quản lý và cấu hình. Các luồng này được kết hợp kiểm thử tự động và chạy thực tế trên thiết bị.

4.6.1 Đăng nhập và phân tách chức năng theo vai trò

Hình 4.1 gồm đăng nhập và hai dashboard. Kết quả xác thực xác định vai trò và nhóm chức năng hiển thị; tầng dịch vụ tiếp tục kiểm quyền đối với từng thao tác. Đổi tài khoản cập nhật phiên, xóa token REST và ngắt OneIoT khi đăng xuất.

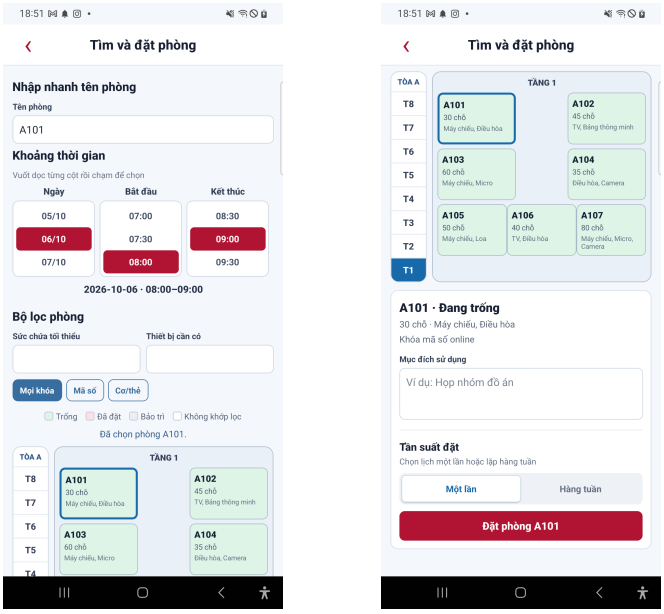
Màn hình đăng nhập sử dụng các trường tài khoản, mật khẩu và thông báo lỗi. Kiểm thử xác thực xác nhận các ô bắt buộc, sai mật khẩu, chuyển biểu mẫu và mở chức năng đúng vai trò (TC01, TC17 và D01–D03).



Hình 4.1: Đăng nhập, chức năng User và Admin (trái sang phải)

4.6.2 Tìm và gửi yêu cầu trên cùng màn hình

Hình 4.2 giữ hai bước: chọn tầng/ngày/giờ, sức chứa/thiết bị/loại khóa; rồi chọn phòng, nhập mục đích và gửi. Ứng dụng tải danh mục và mức chiếm chỗ để hỗ trợ lựa chọn; khi gửi yêu cầu, tầng nghiệp vụ kiểm lại lịch và các quy tắc trước khi ghi. Tám tầng là dữ liệu demo.

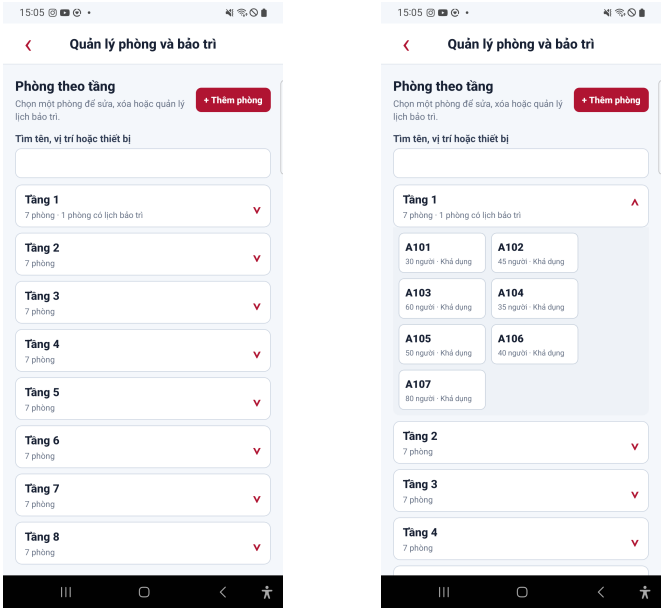


Hình 4.2: Chọn tiêu chí, chọn phòng và gửi yêu cầu (trái sang phải)

4.6.3 Quản lý phòng và bảo trì theo tầng

Hình 4.3 gồm danh sách tầng thu gọn có số phòng/chỉ báo bảo trì và tầng đã mở với thông tin từng phòng. Các tầng có thể mở/thu gọn; trạng thái phòng được tổng hợp từ danh mục, lượt đặt, yêu cầu và lịch bảo trì.

Admin chọn phòng để sửa/xóa, xem hoặc lập/hủy lịch bảo trì. Service chặn xóa phòng còn booking hoạt động; phòng mẫu đã xóa được lưu dấu để không tự tái xuất hiện. Yêu cầu bảo trì phải qua tiếp nhận/lên lịch mới chặn phòng; ghi chú chỉ thông báo admin (UC08, UC13; TC02, TC23).

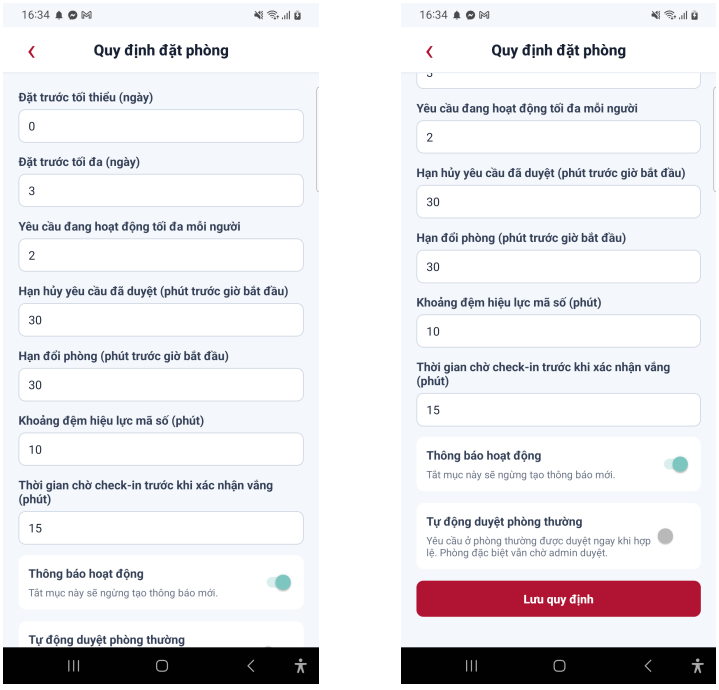


Hình 4.3: Danh sách tầng và phòng khi mở tầng (trái sang phải)

4.6.4 Cấu hình quy tắc đặt phòng

Hình 4.4 thể hiện các tham số đặt phòng và công tắc quản trị trên ứng dụng Android. Phần trên gồm cửa sổ đặt trước, hạn mức, hạn hủy/đổi, đệm PIN và ân hạn check-in; phần dưới gồm thông báo hoạt động, tự duyệt phòng thường và nút lưu. Các giá trị được lưu tập trung để dịch vụ nghiệp vụ áp dụng nhất quán. Cấu hình minh họa có min bằng 0, cho phép đặt cùng ngày; giá trị mặc định là 1 ngày. Khi bật tự duyệt, phòng thường được duyệt ngay sau kiểm tra hợp lệ, còn phòng đặc biệt vẫn chờ admin. Kết quả nghiệp vụ được trình bày tại TC07 và TC24.

Form lọc ký tự không phải số; repository vẫn kiểm số nguyên không âm, min không vượt max, hạn mức ít nhất một và quyền admin. Kiểm thử cấu hình xác nhận việc từ chối giá trị vi phạm và thao tác không đủ quyền (TC16).



Hình 4.4: Tham số đặt phòng (trái), công tắc thông báo và tự duyệt (phải)

4.7 Cấu hình, cài đặt và tái lập bản chạy

Hình 3.4 ở Chương 3 đặt rõ các nơi chạy: APK trên điện thoại, backend/H2 trên máy chủ và kết nối MQTT OneIoT tới khóa. Bảng 4.6 phân biệt điều kiện sử dụng tương ứng.

Bảng 4.6: Chế độ chạy, phụ thuộc và phạm vi dữ liệu

Nhánh	Điều kiện	Hành vi / giới hạn
Local	Cờ REST tắt; APK đã có bundle	Đăng nhập và nghiệp vụ dùng Async Storage riêng trên một máy; không cần booking-server. Không tự tải dữ liệu này lên server khi đổi cờ.
REST	Cờ true; backend/URL có thể truy cập từ điện thoại	Repository gọi API, token theo phiên, dữ liệu ở backend. Bản source đang bật nhánh này; URL tunnel trong source là địa chỉ thử nghiệm.
OneIoT	Internet, token Tools trong phiên và khóa gắn với phòng phù hợp	Kết nối MQTT trực tiếp qua native Android, độc lập với cờ REST. PIN được tạo/lưu trước khi gửi; app quản lý metadata gửi lệnh và sự kiện khóa.

4.7.1 Cấu hình và dữ liệu khởi tạo

Cấu hình tại *code/booking_classroom/src/core/config/runtimeFlags.ts* dùng *ENABLE_REMOTE_SYNC=false* cho local, giá trị true cho REST trước khi build; mã nguồn hiện bật REST. Thay đổi cờ hoặc URL được đưa vào JavaScript bundle khi đóng gói lại APK. Local khởi tạo admin/1 và user/2; dữ liệu mẫu chỉ được bổ sung khi kho chưa có, nên cài đè giữ cấu hình và dữ liệu đã lưu.

4.7.2 Cài đặt và đóng gói Android

Tại *code/booking_classroom/*, *npm ci* cài phụ thuộc theo lockfile. Trên Windows, *npm run build:android:release* tạo bundle, xử lý đường dẫn Unicode qua ổ tạm và gọi Gradle. Trên macOS/Linux, JavaScript được bundle trước khi chạy *sh ./gradlew assembleRelease* trong thư mục android; cấu hình Release hiện bỏ bước bundle tự động. Quy trình lệnh đầy đủ nằm tại mục A.4 và README gốc repository.

APK đầu ra là *android/app/build/outputs/apk/release/app-release.apk*. Sau khi điện thoại cho phép gỡ lỗi USB, cài và mở ứng dụng bằng:

```
adb devices -l
adb install -r android/app/build/outputs/apk/release/app-release.apk
adb shell am start -n com.bookingclassroom/.MainActivity
```

Cài đè giữ dữ liệu khi chữ ký tương thích. Bản Release đã đóng gói JavaScript chạy độc lập Metro; chế độ REST cần kết nối backend, còn OneIoT cần mạng và phiên khóa hợp lệ.

4.7.3 Backend và kết nối phát triển

Tại *code/booking-server/*, *mvn spring-boot:run* khởi động dịch vụ cổng 8080 với H2 file. Kết nối qua USB dùng API base URL *http://127.0.0.1:8080* và *adb reverse tcp:8080 tcp:8080*; kết nối Wi-Fi dùng địa chỉ LAN của máy chủ. URL tunnel là địa chỉ tạm thời, phụ thuộc phiên dịch vụ đang chạy.

Bản Debug dùng *npm start* cho Metro và *npm run android*; *adb reverse tcp:8081 tcp:8081* chuyển tiếp cổng Metro qua USB. Cổng 8081 phục vụ JavaScript phát triển, cổng 8080 phục vụ REST. Cơ sở dữ liệu H2 từ xa là lựa chọn bổ sung qua biến BOOKING_DB; bản local không phụ thuộc cấu hình này.

CHƯƠNG 5. KIỂM THỬ VÀ ĐÁNH GIÁ

5.1 Cấu hình và phương pháp kiểm thử

Bảng 5.1: Môi trường, dữ liệu và phạm vi kết quả

Nhóm kiểm tra	Điều kiện và dữ liệu	Kết quả ghi trong hồ sơ
Jest app	Nhánh main; macOS 14.8.9/arm64; Node 22.17.0, Jest 29.7.0. Có 8 suite auth/booking/UI và auto approval; storage/native được mock.	50/50 ca đạt, gồm 11 ca tự duyệt.
Quality suite	Cùng máy và Node/Jest; 17 suite unit, component, integration; mock fetch/native/Async Storage.	398/398 ca đạt. Unit 259, component 58, integration 81.
Quyền thông báo	Bộ bổ sung riêng: 1 suite, 5 ca; mock API 31/33, quyền cho phép/từ chối và native bridge.	5/5 ca đạt, kiểm logic quyền và native bridge bằng mock.
Backend/H2	2 ca tích hợp, H2 trong RAM, reset context giữa ca; tài khoản/booking và client cũ thiếu trường mới.	2/2 ca tích hợp đạt với cơ sở dữ liệu H2 trong RAM.
Android thật	Samsung Galaxy M21/SM-M215F; Android 12, API 31; arm64-v8a; MemTotal 3.656.308 KiB (khoảng 3,49 GiB). ADB 37.0.1 qua USB.	APK Release local, package com.bookingclassroom, versionName 1.0/versionCode 1; khởi động độc lập Metro.
Kiểm tra tĩnh	TypeScript 5.9.3 và ESLint 8.57.1 theo dependency đã cài; chạy trên cùng source với Jest.	tsc –noEmit và eslint . đều kết thúc mã 0.

5.1.1 Dữ liệu thử và kiểm soát điều kiện

Máy thật dùng admin/1, user/2 và tài khoản mới *qa0811*, mật khẩu thử 8 ký tự, mã khôi phục 8 chữ số. Hồ sơ thử: *RoomHub QA Persist 0811*, *qa0811@example.invalid*, đơn vị *QA RoomHub 2026*. Không sửa dữ liệu tài khoản cũ; công tắc thông báo được chụp trước/sau và trả về bật. Tài khoản thử được giữ lại để tái lập kết quả.

Bộ kiểm thử chất lượng dùng mốc 08:00 của một ngày giả lập cố định. Các ca đặt phòng dùng ngày tính từ mốc này, trạng thái booking và cấu hình được seed theo từng tình huống. Một số ca chủ động đổi mốc thời gian để kiểm ranh giới tháng, hạn hủy hoặc ân hạn vắng mặt.

Trước mỗi ca liên quan, kho Async Storage giả lập được đặt lại; lượt đặt và cấu hình được khởi tạo theo tình huống. Cách tổ chức này tách các ca có cùng user/phòng, giúp xác định lỗi do điều kiện nghiệp vụ thay vì dữ liệu sót từ ca trước. Dữ liệu có cả hai vai trò, phòng mã số/cơ/thẻ, lượt chờ/đã duyệt/hủy và thời điểm nằm trong/ngoài khoảng cho phép.

Hai ca Spring Boot dùng HTTP qua TestRestTemplate và H2 trong RAM. Ca thứ nhất đi qua hồ sơ, chuỗi booking, duyệt, gắn khóa, lưu metadata PIN và tách dữ liệu theo tài khoản; ca thứ hai kiểm tự duyệt và client cũ không gửi hai trường mới. Môi trường Spring được khởi tạo lại sau mỗi phương thức để tách dữ liệu giữa hai ca; mỗi ca kiểm tra một chuỗi nhiều thao tác HTTP.

5.1.2 Các phương pháp kiểm thử

Quy trình kết hợp kiểm thử logic, giao diện, API, tích hợp và vận hành thiết bị. Bảng 5.2 trình bày nội dung và vai trò của từng phương pháp.

Bảng 5.2: Phương pháp và vai trò trong kiểm thử

Phương pháp	Nội dung kiểm tra	Vai trò
Unit/component	Quy tắc dữ liệu, phân quyền, thay đổi state và phản hồi giao diện với đầu vào xác định.	Kiểm tra từng thành phần và phát hiện lỗi nghiệp vụ/giao diện.

Phương pháp và vai trò trong kiểm thử (tiếp)

Phương pháp	Nội dung kiểm tra	Vai trò
Integration local	Chuỗi thao tác giữa service, dữ liệu booking, quyền, PIN và thông báo; mô phỏng sự kiện khóa.	Kiểm tra sự phối hợp giữa các module và các nhánh ngoại lệ.
Hợp đồng API	Method/path, payload, token Bearer, mã lỗi và cấu trúc phản hồi.	Giữ thống nhất giao tiếp giữa ứng dụng và backend.
Spring Boot/H2	Luồng HTTP, quyền theo token, ghi/đọc dữ liệu và tương thích cấu hình.	Kiểm tra nghiệp vụ server và giao dịch lưu trữ.
Android và SmartLock	Cài/khởi động, đăng nhập, đặt/duyệt phòng, cấp PIN, kết nối khóa, sự kiện và thông báo.	Xác nhận các luồng sử dụng và tích hợp app/khóa trong vận hành thực tế.
Ảnh giao diện	Màn hình, thao tác và trạng thái kết quả của các chức năng.	Minh họa trải nghiệm người dùng cùng kết quả thực nghiệm.

Các bộ kiểm thử có thể cùng kiểm một quy tắc ở nhiều mức, nên tổng số ca không tương ứng với số chức năng độc lập. Chỉ tiêu độ phủ mã được tách khỏi kết quả đạt/không đạt của các ca chức năng.

5.2 Ca kiểm thử chức năng và kết quả thực tế

Bảng 5.3 tổng hợp ca nghiệp vụ; Bảng 5.4 ghi nhận thao tác Android [14, 15, 16]. Nhóm xác nhận hoàn thành các chức năng app–SmartLock qua vận hành thực tế [17].

Bảng 5.3: Ca tiêu biểu, kỳ vọng, thực tế và kết luận

ID / dữ liệu	Kỳ vọng	Kết quả thực tế	Kết luận
TC01: admin/1, user/2; user/wrong	Hợp lệ mở đúng vai trò; sai không tạo phiên UI.	Sai mật khẩu báo lỗi; admin/user hợp lệ mở đúng dashboard trên Android.	Đạt: Android và mock.
TC02: lịch phòng chồng giờ, bảo trì và vượt hạn mức	Từ chối tạo; khoảng liền kề không chồng.	Từ chối trùng phòng/user, bảo trì và vượt hạn mức; nhận hai khoảng chạm biên.	Đạt: quality.
TC03: đã duyệt, phòng PIN, đúng user/quyền	Mã 7 số; hiệu lực theo đệm; chặn mã thứ hai khi PIN trước chưa thu hồi.	PIN đủ 7 số, hiệu lực đúng đệm; chặn mã thứ hai khi PIN cũ chưa thu hồi.	Đạt: local/mock.
TC04: HTTP 401 / JSON hỏng / mất mạng	Xóa token với 401; hiển thị lỗi phù hợp; không fallback local.	401 xóa token; JSON hỏng/mất mạng báo lỗi; vẫn giữ nguồn REST.	Đạt: HTTP mock.
TC05: request vượt 12 giây giả lập	Abort request và thông báo timeout; clear timer.	Đồng hồ giả lập tới 12 giây: request bị abort, trả lỗi timeout và dọn timer.	Đạt: timer giả lập.
TC06: thông báo admin khi user đang đăng nhập	Lưu đúng người nhận; không hiện nhầm banner.	Thông báo được lưu cho admin; khi user đang đăng nhập, banner dành cho admin không xuất hiện.	Đạt: quality.
TC07: bật tự duyệt; phòng thường/phòng đặc biệt	Thường APPROVED, special PENDING; vẫn kiểm booking rules.	Phòng thường chuyển APPROVED; phòng đặc biệt giữ PENDING; các yêu cầu vi phạm quy tắc vẫn bị chặn.	Đạt: 11 ca app.
TC08: từ chối thông báo; phân biệt API 31 và 33+	UI vẫn dùng được; quyền bị chặn không gửi native trên API 33+.	API 33 giả lập chặn native khi bị từ chối quyền; API 31 tắt thông báo vẫn lưu hồ sơ và hiện banner.	Đạt: mock/API 31.
TC09: lưu hồ sơ qa0811, force-stop và mở lại	App mở không cần Metro; đăng nhập lại đọc đúng dữ liệu đã lưu.	Sau force-stop, app mở với PID mới và yêu cầu đăng nhập; họ tên, email và đơn vị giữ nguyên.	Đạt: Android.
TC10: ứng dụng kết nối và sử dụng SmartLock	App kết nối OneIoT, cấp mã và thao tác khóa đúng luồng nghiệp vụ.	Theo kết quả vận hành nhóm xác nhận, app kết nối OneIoT, cấp mã và thực hiện luồng sử dụng SmartLock.	Đạt: nhóm nghiệm thu.

5.2.1 Kiểm thực tế trên Android

D01–D06 dùng APK local mục 5.1, có ảnh PNG/XML và *persistence-result.json*. Hồ sơ: *docs/reports/final/LTUD/minh_chung/kiem_thu/* [16].

Bảng 5.4: Bước thử, kỳ vọng và quan sát trên máy thật

ID / bước và dữ liệu	Kỳ vọng	Thực tế / minh chứng / kết luận
D01 – Đăng nhập user với mật khẩu wrong.	Báo lỗi; vẫn ở trang đăng nhập.	Hiện Sai tài khoản hoặc mật khẩu; không mở dashboard. Ảnh/XML 04-wrong-password-rejected. Đạt.
D02 – Đăng nhập user/2, đăng xuất rồi admin/1.	Phân biệt giảng viên và quản trị viên.	UI hiện GIẢNG VIÊN và QUẢN TRỊ; menu tương ứng. Minh chứng 07-user-dashboard-clean và 45-admin-dashboard. Đạt.
D03 – Đăng ký qa0811 với hai mật khẩu trùng, email thử và mã khôi phục hợp lệ; đăng nhập.	Tạo tài khoản giảng viên; đọc được hồ sơ đã nhập.	Thông báo đăng ký thành công, dashboard đúng user, hồ sơ ban đầu đúng. Minh chứng 13, 17, 19. Đạt.
D04 – Trong hồ sơ qa0811, thay họ tên bằng một dấu cách rồi lưu.	Chặn giá trị trống sau trim.	Hiện Họ và tên không được để trống; sửa lại dữ liệu hợp lệ lưu được. Minh chứng 23-profile-validation-error và 31. Đạt.
D05 – Tắt Hiện thông báo trong Android; mở app, lưu hồ sơ thử.	Thao tác và phản hồi trong app vẫn dùng được.	Cài đặt ghi Thông báo từ ứng dụng này bị chặn; app vẫn báo đã lưu và hiện banner. Minh chứng 28, 30; bật lại ở 39. Đạt trên API 31.
D06 – Lưu ba trường hồ sơ mục 5.1; force-stop, kiểm tiến trình dừng, mở lại, đăng nhập qa0811 và đọc hồ sơ.	Mở độc lập Metro; dữ liệu đã lưu giữ nguyên.	Không còn tiến trình sau dừng, PID đổi khi mở; ba trường trước/sau bằng nhau. Minh chứng 31, 32, 37 và JSON đối chiếu. Đạt với hồ sơ local.

Trên Android API 31, D05 kiểm tra công tắc thông báo hệ thống. Bộ bổ sung mô phỏng API 33 để kiểm logic xin quyền, từ chối quyền và hiển thị banner; D06 kiểm tra dữ liệu hồ sơ trước và sau khi đóng/mở ứng dụng.

5.2.2 Ca chi tiết theo nhóm nghiệp vụ

Các ca TC11–TC24 làm rõ đầu vào và ngoại lệ của các luồng còn lại. TC11–TC23 tổng hợp kết quả từ bộ quality 398 ca đạt với storage/native được mô phỏng [16]; TC24 dùng kết quả tích hợp H2 [14]. Danh mục tệp kiểm thử và kết quả gốc được lưu trong hồ sơ minh chứng. Các ca Android được trình bày riêng ở Bảng 5.4.

Bảng 5.5: Ca nghiệp vụ chi tiết và kết quả

ID / bước và dữ liệu	Kỳ vọng	Kết quả thực tế / kết luận
TC11 – Tạo lượt hợp lệ với tự duyệt tắt; kiểm thông báo sau ghi.	PENDING; admin nhận thông báo yêu cầu mới, mục đích được chuẩn hóa.	Lượt được lưu PENDING với mục đích đã bỏ khoảng trắng thừa; thông báo yêu cầu mới được ghi cho admin. Đạt (quality).
TC12 – Dịch ngày đặt 0/1/2/3/4 ngày khi cấu hình 1–3; sau đó seed cửa sổ 2–5 ngày.	Chỉ 1/2/3 hợp lệ theo mặc định; với cấu hình mới, ngày 1 bị chặn và ngày 5 được nhận.	Với cửa sổ 1–3, ngày 0/4 bị từ chối và ngày 1/2/3 được nhận; đổi sang 2–5 chặn ngày 1, nhận ngày 5. Đạt (quality).
TC13 – Yêu cầu PENDING, phòng cơ/thẻ, người khác, quyền chưa cấp hoặc booking đã kết thúc thử tạo PIN.	Từ chối theo trạng thái, loại phòng, người đặt/quyền và thời hạn tương ứng.	Các trường hợp sai trạng thái, sai loại khóa, khác chủ/quyền hoặc đã hết giờ đều trả lỗi tương ứng và không tạo PIN. Đạt (quality).

Ca nghiệp vụ chi tiết và kết quả (tiếp)

ID / bước và dữ liệu	Kỳ vọng	Kết quả thực tế / kết luận
TC14 – Thu hồi quyền user–phòng rồi tạo PIN; cấp lại quyền khi cần.	Sau thu hồi, user bị chặn tự tạo PIN; cấp lại tái sử dụng bản ghi quyền. Việc tạo mã còn phải thỏa điều kiện booking/PIN riêng.	Sau thu hồi, kiểm tra quyền trả false và tạo PIN bị chặn; cấp lại kích hoạt cùng định danh bản ghi quyền. Đạt (quality).
TC15 – Đổi booking sang phòng PIN khác khi khóa duy nhất vẫn gắn phòng cũ.	Ghi phòng mới, đánh dấu PIN cũ thu hồi; không tạo mã cho phòng chưa gắn SmartLock.	Lượt trỏ sang phòng mới; PIN cũ có dấu thu hồi và quyền phòng mới được cấp. Tạo PIN tại phòng chưa gắn khóa trả lỗi. Đạt (quality).
TC16 – User thử tạo phòng/tài khoản hoặc lưu cấu hình quản trị.	Service từ chối; không dựa riêng vào việc menu bị ẩn.	User nhận lỗi không đủ quyền khi ghi phòng, tạo tài khoản hoặc cập nhật cấu hình; dữ liệu quản trị không được ghi. Đạt (quality).
TC17 – Tài khoản bị khóa đăng nhập; đổi/khôi phục mật khẩu với input sai và đúng.	Không tạo phiên cho tài khoản khóa; mật khẩu cũ sai sau đổi; mã khôi phục sai bị từ chối.	Tài khoản khóa không tạo phiên; sau đổi, mật khẩu cũ không còn hợp lệ; mã khôi phục sai bị từ chối. Đạt (quality).
TC18 – Admin tạo/sửa/xóa phòng; thử xóa khi còn booking hoạt động và thử đọc lại phòng mẫu đã xóa.	Chuẩn hóa dữ liệu, giữ định danh khi sửa, chặn xóa khi còn lượt hoạt động; phòng mẫu đã xóa không tự tái xuất hiện.	Sửa phòng giữ định danh; xóa phòng có lượt hoạt động bị chặn. Phòng mẫu đã xóa không xuất hiện lại khi đọc danh mục. Đạt (quality).
TC19 – Đề xuất/đề xuất lại lịch nhận chìa; sai lượt phản hồi; thiếu địa điểm; nhập người nhận hộ.	Hai bên xác nhận đúng lượt; admin phải có địa điểm; lịch tương lai trước giờ dùng; người nhận hộ có đủ tên và mã.	Lịch chuyển AGREED khi hai bên chấp nhận đúng lượt; lịch sử đề xuất và người nhận hộ được lưu. Sai lượt/thiếu địa điểm trả lỗi. Đạt (quality).
TC20 – Đổi phòng sai loại khóa, thiếu sức chứa/thiết bị, phòng đã bị giữ hoặc quá hạn.	Chặn thay thế không phù hợp; chỉ đổi lượt đã duyệt khi đủ điều kiện và ghi lịch sử.	Phòng sai loại khóa, thiếu sức chứa/thiết bị, đã bị giữ hoặc quá hạn bị từ chối; ca hợp lệ cập nhật phòng và lịch sử. Đạt (quality).
TC21 – Đặt chuỗi tuần với một tuần sau xung đột; sửa/hủy chuỗi theo người sở hữu và hạn chót.	Không tạo dở chuỗi khi một lượt xung đột; chỉ sửa lượt tương lai PENDING; hủy kiểm toàn bộ lượt chịu cutoff.	Một tuần xung đột làm thao tác tạo/sửa bị từ chối toàn bộ; không ghi dở chuỗi. Sửa và hủy chỉ tác động các lượt đủ điều kiện. Đạt (quality).
TC22 – Đánh dấu no-show trước/tại ngưỡng 15 phút hoặc sau khi đã có check-in.	Chỉ admin, đủ ân hạn và chưa có bằng chứng có mặt; no-show giải phóng khung giờ, đánh dấu PIN thu hồi trong dữ liệu.	Trước ân hạn hoặc đã check-in bị từ chối; đúng ngưỡng được ghi NO_SHOW, đánh dấu PIN thu hồi và cho phép đặt lại khung giờ. Đạt (quality).
TC23 – Chủ lượt APPROVED đang sử dụng gửi ghi chú hoặc báo sự cố.	Ghi chú gửi admin nhưng không tạo maintenance; yêu cầu bảo trì qua bước lên lịch mới chặn phòng.	Ghi chú tạo thông báo admin, kho bảo trì không tăng; yêu cầu bảo trì chỉ sinh lịch chặn phòng sau khi admin lên lịch. Đạt (quality).
TC24 – Client cũ PUT cấu hình/phòng thiếu autoApprovalEnabled hoặc requiresApproval.	Giữ giá trị đã có; phòng thường tự duyệt, phòng đặc biệt vẫn PENDING tới khi admin xử lý.	Cập nhật thiếu hai cờ mới giữ nguyên giá trị đã lưu; phòng thường APPROVED, phòng đặc biệt PENDING. Đạt theo hồ sơ 2/2 ca H2.

Vòng đời PIN được quản lý bằng revokedAt và trạng thái booking. Quyền user–phòng là bản ghi riêng; thao tác hủy/no-show cập nhật lượt đặt và PIN tương ứng. Việc tạo PIN tiếp theo tuân theo trạng thái thu hồi của mã hiện có.

5.2.3 Truy vết một chuỗi kiểm thử tích hợp

Chuỗi tích hợp phòng PIN bắt đầu bằng đăng ký tài khoản gv.an, gắn khóa cho room-a101 và tạo booking 09:00–11:00 vào ngày kế tiếp mốc giả lập. Sau admin duyệt, test xác nhận quyền phòng; tạo PIN khi OneIoT chưa kết nối phải có mã bảy số nhưng chưa có lockDeliveredAt. Khi

mock kết nối chuyển online, test kiểm gửi đúng phòng, ID mật khẩu, epoch từ 08:50 tới 11:10 và không gửi lại mã đã ghi nhận gửi.

Bước tiếp theo đưa bản tin traitUnlockTempPwd lúc 08:55 và traitLocked lúc 10:58 vào parser để kiểm checkedInAt/checkedOutAt. Sau có check-in, yêu cầu đánh dấu vắng mặt phải bị từ chối. Chuỗi này kết nối các module booking, quyền phòng, PIN, MQTT adapter và thông báo; lớp mạng/native được mock để kiểm soát đầu vào và tái hiện các tình huống kết nối.

Ca riêng giả lập MQTT timeout, kiểm lockDeliveryError và lần kết nối sau gửi thành công phải xóa lỗi; qua đó kiểm trạng thái chờ/lỗi và gửi lại trong dữ liệu ứng dụng.

Kết quả từng bộ được lưu dưới dạng JSON và nhật ký; ảnh PNG/XML ghi nhận thao tác Android và trạng thái dữ liệu trước/sau. Các lệnh tái lập tập trung tại mục A.4.2; hồ sơ chi tiết được dẫn trong tài liệu [16].

5.3 Phạm vi đánh giá chất lượng định lượng

Đánh giá hiện tại tập trung vào tính đúng chức năng và khả năng vận hành Android–SmartLock. Các phép đo thời gian khởi động, độ trễ đầu cuối và ổn định dài hạn không được thực hiện trong đợt đánh giá chức năng; Bảng 5.6 nêu điều kiện đo và phạm vi tương ứng.

Bảng 5.6: Phương pháp và phạm vi đánh giá định lượng

Chỉ tiêu	Phương pháp / điều kiện cần	Kết quả và lý do
Cold-start Android	Đo từ lệnh khởi chạy tới giao diện sẵn sàng; ghi thiết bị/OS, N lần và log.	Không thực hiện phép đo thời gian trong phạm vi đánh giá chức năng.
Phản hồi đặt/duyệt	Đo end-to-end trên thiết bị với cùng server/mạng/dữ liệu và N lần.	Không thực hiện phép đo độ trễ định lượng trong phạm vi đánh giá chức năng.
Ổn định dài hạn	Lặp kịch bản có quy trình và số lần chạy; theo dõi crash, listener và bộ nhớ.	Không thực hiện phép đo độ ổn định dài hạn trong phạm vi đánh giá chức năng.

5.4 Đối chiếu yêu cầu, hạn chế và rủi ro

FR01–FR08 đã được triển khai trong sản phẩm. Kết quả kiểm thử tự động và vận hành thực tế của nhóm xác nhận các luồng đăng nhập, đặt/duyệt phòng, cấp quyền, PIN/SmartLock, điểm danh, bảo trì và quản trị đáp ứng chức năng của đề tài [17].

Các hạn chế và ràng buộc bảo mật đã được tập trung tại mục 3.5 và các giải pháp ở Chương 3. Ưu tiên khi mở rộng vận hành: xác thực/chứng thư, lưu an toàn, phản hồi/thu hồi PIN và dịch vụ nhận sự kiện bền vững.

Các lỗi phòng mẫu tự tái xuất hiện, từ chối đăng ký khi bỏ trống mã khôi phục tùy chọn, banner sai người nhận và cập nhật thiếu cờ tự duyệt đã được sửa và có kiểm thử hồi quy [14]. Với đăng ký từ ứng dụng, mã để trống được thay bằng *000000*, còn mã tự nhập phải có ít nhất 4 ký tự. Khi quên mật khẩu, người dùng nhập tên đăng nhập và mã khôi phục để đặt mật khẩu mới; nhập sai mã bị từ chối. Các hạn chế kỹ thuật được tổng hợp để định hướng mở rộng và gia cố sản phẩm.

CHƯƠNG 6. TỔ CHỨC NHÓM VÀ TỔNG KẾT

6.1 Quá trình thực hiện và các mốc chính

Quá trình thực hiện được chia thành các giai đoạn xây dựng nền tảng, hoàn thiện nghiệp vụ, kết nối dữ liệu, kiểm thử và hoàn thiện tài liệu.

Bảng 6.1: Giai đoạn thực hiện và sản phẩm

Giai đoạn	Sản phẩm / kết quả	Thành viên liên quan
Xây dựng nền tảng	Khung ứng dụng, xác thực, danh mục phòng, PIN và điều hướng theo vai trò.	Hoàng Văn Tùng.
Hoàn thiện nghiệp vụ	Đặt lặp, duyệt yêu cầu, thông báo và xử lý vắng mặt; tích hợp với quy tắc đặt phòng.	Nguyễn Đình Phúc; phối hợp với nhóm.
Kết nối dữ liệu	API và trao đổi dữ liệu app/server, lọc phòng và kiểm tra mức chiếm chỗ.	Đỗ Thùy Linh; phối hợp tích hợp app/server.
Kiểm thử và tích hợp	Bộ kiểm thử chất lượng, tự động duyệt và kiểm tra hồi quy các luồng nghiệp vụ.	Nguyễn Việt Dũng, Hoàng Văn Tùng; phối hợp với nhóm.
Hoàn thiện tài liệu	Tổ chức controller và tiện ích dùng chung; sơ đồ thiết kế, hướng dẫn sử dụng và báo cáo.	Đỗ Thùy Linh, Nguyễn Đình Phúc và các thành viên.

6.2 Phân công và đóng góp

Bảng 6.2 tổng hợp phần mã nguồn và kiểm thử theo thành viên, cùng tỷ lệ đóng góp nhóm đã thống nhất. Tập thống kê gồm App.tsx, src TypeScript/TSX, native Kotlin, backend Java và các test JS/TS/Java; không tính lockfile, hình ảnh, binary, build và tài liệu. Tổng cộng có 115 tệp và 15.213 dòng không rỗng, kể cả comment.

Bảng 6.2: Mã nguồn, kiểm thử và tỷ lệ đóng góp

Thành viên	Code / test	Tổng dòng	Đóng góp
Đỗ Thùy Linh	1.988 / 37	2.025	26%
Hoàng Văn Tùng	8.147 / 787	8.934	28%
Nguyễn Việt Dũng	196 / 3.033	3.229	26%
Nguyễn Đình Phúc	726 / 299	1.025	20%
Tổng	11.057 / 4.156	15.213	100%

Số dòng hỗ trợ minh chứng phần triển khai, còn tỷ lệ đóng góp phản ánh cả thiết kế, tích hợp, kiểm thử, tài liệu và phối hợp. Sản phẩm của từng thành viên được nêu ở Phụ lục B. Phương pháp thống kê dùng quy thuộc dòng theo tác giả trong Git [18]; dữ liệu chi tiết và script tái lập được lưu tại *minh_chung/*.

6.3 Kết quả so với mục tiêu

G01 hoàn thành luồng đặt phòng và kiểm các quy tắc thời gian, xung đột, bảo trì, hạn mức. G02 hoàn thành duyệt, cấp quyền, PIN và hẹn nhận chìa; ứng dụng đã vận hành thực tế cùng SmartLock. G03 có bản Android hoạt động theo vai trò, giao diện và thông báo phục vụ các tác vụ chính. G04 có mã nguồn, README, bộ kiểm thử và hồ sơ bàn giao để cài đặt, sử dụng và phát triển tiếp.

Quy trình tiếp cận phòng được liên kết từ yêu cầu hợp lệ, phê duyệt và cấp quyền đến tạo/gửi mã, sử dụng SmartLock và cập nhật sự kiện. Giao diện phản ánh trạng thái của từng bước để người dùng theo dõi thao tác.

6.4 Bài học kỹ thuật và hướng phát triển

Trong quá trình triển khai, nhóm nhận thấy việc giữ phiên ở thành phần gốc và trạng thái biểu mẫu ở từng màn giúp hạn chế dữ liệu lặp, thuận tiện xử lý đăng nhập/đăng xuất. Giải phóng listener, quản lý loading/error và dọn tác vụ bất đồng bộ giúp ổn định luồng chuyển màn. Kiểm tra nghiệp vụ tại service/server bổ sung bảo vệ cho validation giao diện. Kiểm thử tách storage, HTTP và native giúp tái hiện các tình huống biên; thử nghiệm trên Android và SmartLock bổ sung đánh giá tích hợp. Việc thống nhất model và hợp đồng API giữa các thành viên hỗ trợ tích hợp hai chế độ dữ liệu.

Ba hướng tiếp theo là: (1) gia cố xác thực, mã hóa lưu trữ và kiểm chứng chứng thư; (2) hoàn thiện quản lý vòng đời PIN và tiếp nhận sự kiện khóa bền vững; (3) thử nhiều điện thoại trên server cố định, đo hiệu năng có quy trình và mở rộng dữ liệu nhiều tòa/khóa. Các tiêu chí đánh giá tương ứng là mức bảo vệ dữ liệu, tính đầy đủ của phản hồi khóa và độ trễ/độ ổn định khi tăng số thiết bị, dữ liệu.

TÀI LIỆU THAM KHẢO

[1] Trường Điện – Điện tử, Đại học Bách khoa Hà Nội, *Mẫu báo cáo tổng kết dự án nhóm học phần ET4710 – Lập trình ứng dụng di động*. Tài liệu hướng dẫn học phần: *ET4710_Template_Bao_cao_Nhom_Hoc_vien_React_Native.docx*.

[2] Nhóm RoomHub, *Quy trình nghiệp vụ RoomHub, phiên bản 2.1*. Tài liệu nghiệp vụ nội bộ: *docs/project/references/quy_trinh_nghiep_vu/Quy_trinh_nghiep_vu_RoomHub_v2.1.docx*.

[3] Nhóm RoomHub, *classroom-booking*, mã nguồn và tài liệu dự án trên nhánh main. https://github.com/xwomen1/classroom-booking/tree/main.

[4] Meta, *React Native Documentation*. Phiên bản dự án 0.86.2. https://reactnative.dev/docs/getting-started.

[5] React Team, *Hooks API: useState, useEffect*. https://react.dev/reference/react.

[6] Microsoft, *TypeScript Handbook*. https://www.typescriptlang.org/docs/handbook/intro.html.

[7] React Native Async Storage, *Documentation*. Phiên bản dự án 3.1.1. https://react-native-async-storage.github.io/async-storage/.

[8] OpenJS Foundation, *Jest Documentation*. Phiên bản dự án 29. https://jestjs.io/docs/getting-started.

[9] Spring Team, *Spring Boot Reference Documentation 3.4*. https://docs.spring.io/spring-boot/3.4/index.html.

[10] H2 Group, *H2 Database Engine*. https://www.h2database.com/html/main.html.

[11] OASIS, *MQTT Version 3.1.1*, 2014. https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html.

[12] oneM2M Partners, *Published specifications: MQTT protocol binding*. https://www.onem2m.org/technical/published-specifications.

[13] Android Developers, *Notification runtime permission*. https://developer.android.com/develop/ui/views/notifications/notification-permission.

[14] Nhóm RoomHub, *Báo cáo kiểm thử*. *code/booking_classroom/quality-tests/BAO_CAO_KIEM_THU.md*.

[15] Nhóm RoomHub, *Biên bản kiểm tra ứng dụng trên thiết bị Android*. *code/booking_classroom/evidence/device-verification.txt*.

[16] Nhóm RoomHub, *Hồ sơ kết quả kiểm thử tự động và thực nghiệm Android*. Hồ sơ kèm báo cáo: *minh_chung/kiem_thu/README.md*; kết quả Jest, SHA-256 APK, ảnh PNG và cây giao diện XML.

[17] Nhóm RoomHub, *Xác nhận vận hành ứng dụng và SmartLock*. Hồ sơ kèm báo cáo: *minh_chung/kiem_thu/nghiem_thu_he_thong.md*.

[18] Git Project, *git-blame Documentation*. https://git-scm.com/docs/git-blame. Tài liệu về quy thuộc dòng mã theo tác giả.

PHỤ LỤC A. DANH MỤC MINH CHỨNG VÀ HƯỚNG DẪN DEMO

A.1 Danh mục minh chứng

Kho bàn giao sử dụng nhánh main. Đường dẫn ứng dụng tính từ *code/booking_classroom/*, backend từ *code/booking-server/*, báo cáo từ *docs/reports/final/LTUD/*; các đường dẫn khác tính từ gốc repository.

Bảng A.1: Hồ sơ bàn giao và vị trí

Hạng mục	**Vị trí / liên kết**	**Ghi chú**
Kho mã nguồn	GitHub: xwomen1/classroom-booking, nhánh main.	Ứng dụng, backend, kiểm thử và tài liệu.
README	Gốc repository: *README.md*; ứng dụng và backend: *README.md* tại từng thư mục.	Phụ thuộc, cấu hình, build và cài Android; Java/Maven, H2.
Mã kiểm thử	Ứng dụng: *__tests__/*, *quality-tests/*, *src/modules/auto_approval/__tests__/*.	Bộ kiểm thử và cấu hình chạy riêng.
Báo cáo kiểm thử	Ứng dụng: *quality-tests/BAO_CAO_KIEM_THU.md*; *evidence/device-verification.txt*.	Kết quả tự động, tích hợp H2 và kiểm tra Android.
Hồ sơ thực nghiệm	Báo cáo: *minh_chung/kiem_thu/README.md*.	Môi trường, 50 ca app/398 quality/5 quyền thông báo; 6 ca Android, ảnh/XML và log.
Nghiệm thu hệ thống	Báo cáo: *minh_chung/kiem_thu/nghiem_thu_he_thong.md*.	Kết quả vận hành ứng dụng và SmartLock do nhóm xác nhận.
Ảnh giao diện	Báo cáo: *Hinhve/*hien_tai.png*; ảnh Android trong hồ sơ thực nghiệm.	Minh họa các chức năng tại Chương 4.
Sơ đồ thiết kế	Gốc: *design/RoomHub_design.drawio*; báo cáo: *Hinhve/design/roomhub_bao_cao.drawio*.	Nghiệp vụ, kiến trúc, dữ liệu và trình tự xử lý.
Bản chạy thử	Ứng dụng: *android/app/build/outputs/apk/release/app-release.apk* sau build.	Cài Android theo mục 4.7; chọn local/REST theo môi trường.
Thống kê đóng góp	Báo cáo: *minh_chung/thong_ke_git.json*; *minh_chung/thong_ke_git.py*.	Số liệu quy thuộc dòng mã và script tái lập.

A.2 Kịch bản trình diễn 3–7 phút

Kịch bản sử dụng bản Android đã cấu hình local hoặc REST, tài khoản user/admin, phòng khả dụng và hạn mức còn đủ. Luồng SmartLock sử dụng token hợp lệ và khóa đã gắn đúng phòng. Bảng A.2 tổng hợp thao tác và trạng thái đầu ra dự kiến trong 3–7 phút.

Bảng A.2: Kịch bản demo và kết quả mong đợi

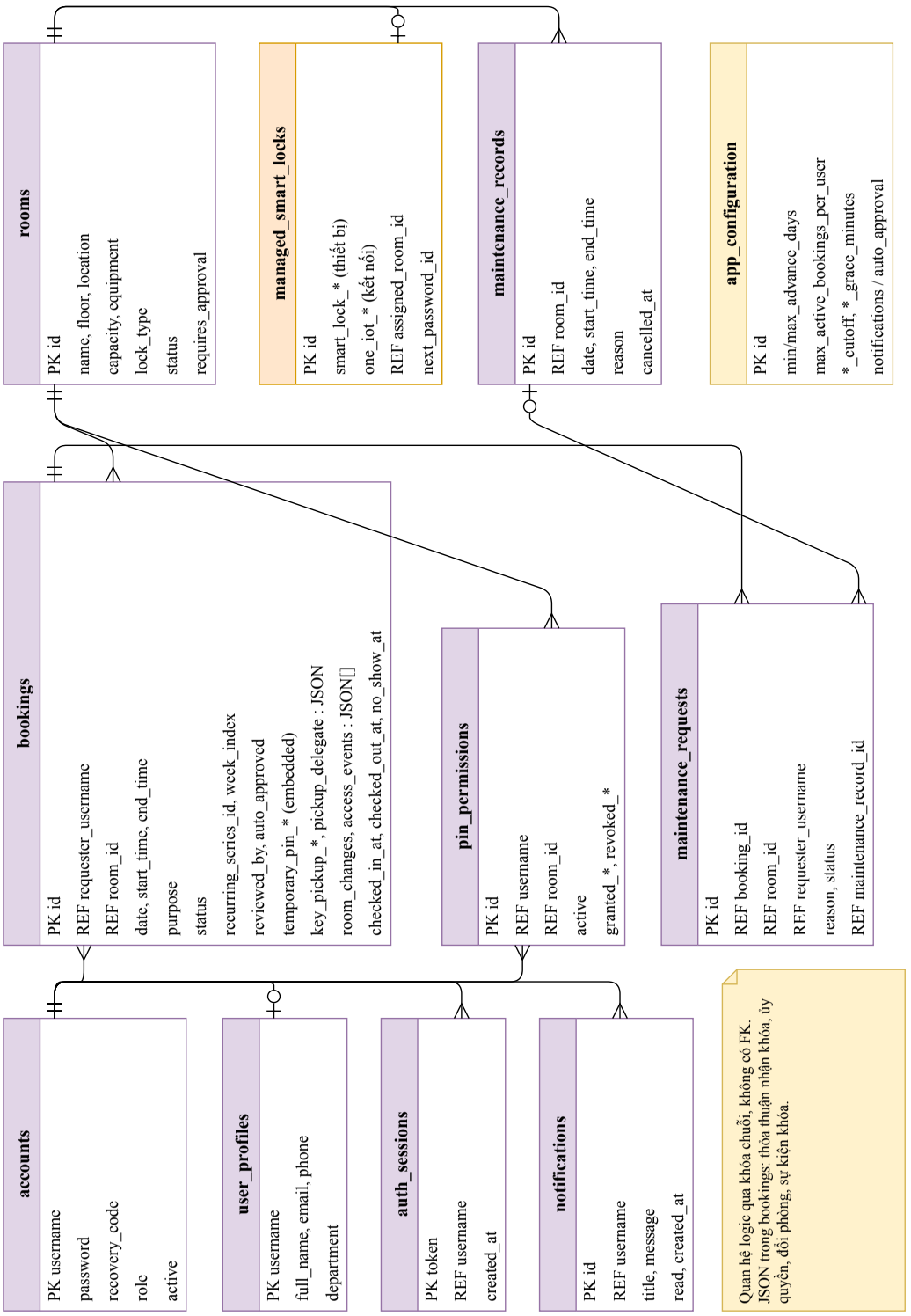
Bước / thời lượng	**Nội dung**	**Kết quả mong đợi**
1 / 30 giây	Giới thiệu bài toán và chế độ dữ liệu của bản chạy; đăng nhập user.	Hiển thị bảng điều khiển User và các chức năng đặt phòng.
2 / 60–90 giây	Chọn phòng/giờ ngày mai trong cửa sổ cho phép, nhập mục đích, gửi yêu cầu.	Tạo PENDING khi tự duyệt tắt; lịch phòng phản ánh khung giờ đã giữ.
3 / 60–90 giây	Đăng nhập admin, duyệt yêu cầu; quay lại user.	APPROVED và quyền tiếp cận đúng phòng. Với PIN, tạo mã 7 số; chờ OneIoT khi chưa kết nối.
4 / 30–60 giây	Thử yêu cầu trùng giờ hoặc ngày ngoài cửa sổ; nhập sai mật khẩu/thiếu mục đích.	Thông báo rõ lý do; không tạo thêm dữ liệu vi phạm.
5 / 30–60 giây	Bật tự duyệt phòng thường và đánh dấu một phòng special; tạo lượt hợp lệ.	Thường được duyệt tự động, special chờ admin. Kết quả tự duyệt được kiểm tra tại TC07 và TC24.
6 / 30 giây	Trình bày mã đã cấp, trạng thái gửi và sử dụng SmartLock trong phiên kết nối.	Ứng dụng thể hiện trạng thái PIN và nhận sự kiện sử dụng khóa.

Khi mất mạng, chế độ REST hiển thị lỗi kết nối và giữ nguồn dữ liệu server đã cấu hình. Các lượt thử được hủy/từ chối và cấu hình được khôi phục trong phạm vi dữ liệu trình diễn.

A.3 Mô hình dữ liệu và nguồn sơ đồ chi tiết

Hình A.1 bổ sung các thuộc tính/quan hệ đã tóm tắt ở Chương 3. PK biểu thị định danh, REF biểu thị các quan hệ tham chiếu logic theo ID.

Nguồn chỉnh sửa sơ đồ được lưu trong *Hinhve/design/roomhub_bao_cao.drawio*. Phân rã chức năng và tổ chức module được tổng hợp tại Bảng 3.2 và Bảng 4.3.



Hình A.1: Mô hình thực thể và quan hệ logic của RoomHub

A.4 Quy trình tái lập kỹ thuật

A.4.1 Đóng gói Release trên macOS/Linux

Các lệnh sau thực hiện tại *code/booking_classroom/*, sau khi cấu hình JDK/SDK, chọn nguồn dữ liệu và chạy *npm ci*. Bước bundle đưa JavaScript và tài nguyên vào APK trước khi Gradle đóng gói:

```
mkdir -p android/app/src/main/assets
node node_modules/react-native/cli.js bundle \
  --platform android --dev false --entry-file index.js \
  --bundle-output android/app/src/main/assets/index.android.bundle \
  --assets-dest android/app/src/main/res
cd android
sh ./gradlew assembleRelease
cd ..
```

Cài APK và mở ứng dụng theo mục 4.7. Trên Windows, script *npm run build:android:release* thực hiện các bước bundle và Gradle tương ứng.

A.4.2 Tái lập kiểm thử

Các bộ ứng dụng, quality, quyền thông báo và kiểm tra tĩnh chạy từ thư mục ứng dụng; dữ liệu kết quả được lưu riêng cho từng bộ:

```
npm test -- --runInBand
npx jest --config quality-tests/jest.config.js --runInBand
npx jest --config ../../docs/reports/final/LTUD/minh_chung/kiem_thu/jest.permission.config.cjs
--runInBand
npx tsc --noEmit
npm run lint
```

Kiểm thử backend dùng *mvn test* tại *code/booking-server/*. Môi trường, dữ liệu và kết quả thực nghiệm Android được lưu trong hồ sơ kiểm thử [16]; lệnh ADB và vị trí ảnh/XML đi kèm hồ sơ.

PHỤ LỤC B. ĐỐI CHIẾU ĐÓNG GÓP THÀNH VIÊN

B.1 Đối chiếu từng thành viên

Bảng này tổng hợp đầu việc, sản phẩm bàn giao và vị trí mã nguồn của từng thành viên. Cột đóng góp dùng tỷ lệ ở Bảng 2; số liệu mã nguồn và kiểm thử được trình bày ở Chương 6. Đường dẫn app tính từ *code/booking_classroom/*; đường dẫn Java tính từ *code/booking-server/src/main/java/com/bookingclassroom/server/*.

Bảng B.1: Đầu việc, sản phẩm và minh chứng thành viên

Thành viên	Đầu việc và kết quả	Vị trí mã nguồn	Đóng góp
Đỗ Thùy Linh	Phòng/lọc, API và chiếm chỗ; tổ chức controller theo chức năng, tiện ích giao giờ.	*src/modules/room_management/*; *src/core/time/intervals.ts*; backend: *api/*.	26%.
Hoàng Văn Tùng	Khung app, điều hướng, xác thực, phòng/booking nền, PIN/native SmartLock; tích hợp và sửa hồi quy.	*App.tsx*; *src/app/*; *src/modules/auth/*; *src/modules/access_control/*; *src/modules/smart_lock/*.	28%.
Nguyễn Việt Dũng	Bộ kiểm thử chất lượng và báo cáo test; tự duyệt phòng thường ở app/server.	*quality-tests/*; *src/modules/auto_approval/*; backend *AutoApprovalPolicy.java*.	26%.
Nguyễn Đình Phúc	Lịch lặp, vắng mặt, màn duyệt/thông báo, sơ đồ và báo cáo.	*src/modules/booking/*; *src/modules/notifications/*; *design/*; báo cáo LTUD.	20%.

Các vị trí mã nguồn và tệp kiểm thử hỗ trợ kiểm tra sản phẩm bàn giao của từng thành viên. Tỷ lệ đóng góp bao gồm cả công việc thiết kế, tích hợp, thử nghiệm, tài liệu và phối hợp.

B.2 Tái lập thống kê

Dữ liệu thống kê được lưu trong *minh_chung/thong_ke_git.json*; script *minh_chung/thong_ke_git.py* hỗ trợ quy thuộc dòng theo tác giả từ lịch sử Git. Hướng dẫn chạy và thông tin kỹ thuật được lưu cùng script để tái lập khi cần.

B.3 Công cụ hỗ trợ và tài nguyên bên ngoài

React Native/React, TypeScript, Async Storage, Spring/JPA/H2, Jest và react-test-renderer được khai báo trong package/lockfile và đối chiếu ở Chương 3. Sơ đồ Draw.io, quy trình nghiệp vụ, mẫu ET4710 [1], ảnh ứng dụng của nhóm và dữ liệu phòng demo được dẫn tại Phụ lục A; ảnh minh họa quá trình sử dụng các chức năng của ứng dụng.

Công cụ AI/Codex hỗ trợ tổ chức module, triển khai, sửa lỗi, kiểm thử và biên soạn tài liệu. Nhóm kiểm chứng kết quả bằng đọc mã nguồn, các bộ kiểm thử, thao tác trên Android thật và rà soát báo cáo. Phạm vi xác minh được nêu ở Chương 5; việc dùng công cụ hỗ trợ không thay thế trách nhiệm kiểm tra và hoàn thiện sản phẩm của nhóm.